



Invited Review

The flexible job shop scheduling problem: A review

Stéphane Dauzère-Pérès^{a,b}, Junwen Ding^c, Liji Shen^d, Karim Tamssaouet^{b,*}^a Mines Saint-Etienne, Univ Clermont Auvergne, CNRS, UMR 6158 LIMOS Gardanne, France^b Department of Accounting and Operations Management, BI Norwegian Business School, Oslo, Norway^c School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, China^d WHU - Otto Beisheim School of Management, Vallendar, Germany

ARTICLE INFO

Article history:

Received 23 August 2022

Accepted 10 May 2023

Keywords:

Scheduling

Flexible job shop

Survey

Criteria

Constraints

ABSTRACT

The flexible job shop scheduling problem (FJSP) is an NP-hard combinatorial optimization problem, which has wide applications in the real world. The complexity and relevance of the FJSP have led to numerous research works on its modeling and resolution. This paper reviews some of the research of the past 30 years on the problem, by presenting and classifying the different criteria, constraints, configurations and solution approaches that have been considered. Recent emerging topics on complex shop scheduling, multi-criteria optimization and uncertain and dynamic environments are discussed. Finally, future research opportunities are proposed.

© 2023 Elsevier B.V. All rights reserved.

1. Introduction

As a complex scheduling problem where each job requires a given series of operations, or routing, to be processed on different machines, the Job-shop Scheduling Problem (JSP) has been studied for decades. The decisions to take are the sequencing decisions of the operations on the machines to minimize a given criterion, the most studied in the literature being the makespan, i.e., the maximum completion time of all jobs. Many solution approaches have been proposed in the literature, including the well-known branch-and-bound approach of [Carlier & Pinson \(1989\)](#) and the shifting bottleneck heuristic of [Adams et al. \(1988\)](#), which both rely on the efficient resolution of single-machine scheduling problems.

At the beginning of the 90s, an important extension of the JSP started to be studied where each operation can be processed on any of the machines in a given subset of the set of machines. This problem was called the Flexible Job shop Scheduling Problem (FJSP) in [Brandimarte \(1993\)](#), the job shop scheduling problem with multi-purpose machines in [Brucker & Schlie \(1990\)](#) and [Hurink et al. \(1994\)](#), the multiprocessor job shop scheduling problem in [Dauzère-Pérès & Paulli \(1997\)](#) and later the parallel-machine jobs shop scheduling problem in [Liu & Kozan \(2009\)](#). The significant additional complexity in the FJSP is related to the fact that not only operations must be sequenced on the machines as in

the JSP, but operations must also be assigned to machines. The assignment decisions prevent, in particular, deriving efficient lower bounds for the FJSP by solving single-machine scheduling problems as for the JSP. Hence, the most efficient branch-and-bound approaches for the JSP cannot be extended to solve the FJSP. Moreover, the flexibility of the machines allows the workload, i.e., the sum of the processing times of the operations assigned to the machines, to be better balanced compared to the JSP. This is emphasized when the processing time of an operation varies from one machine to another.

With mass customization and automation, the FJSP is no longer another extension of the JSP but a shop scheduling problem on its own. This survey also intends to show the diversity of practical settings of scheduling problems that can be modeled by extending the FJSP. Solving scheduling problems is not only important for more effective operational shop control but also to solve tactical or strategic optimization problems such as: Manufacturing equipment selection problem ([Mati et al., 2011](#)), delivery date setting ([Alvarez-Valdes et al., 2005](#)) or process planning ([Özgülven et al., 2010](#)).

This paper does not intend to provide an exhaustive survey of all papers published on the FJSP, as hundreds of references would be needed, with an increased number of papers in the past ten years. Instead, we present what we believe are well-known models and important results, and survey the different criteria, additional constraints and various configurations of the FJSP considered in the literature. In addition, we propose relevant perspectives for future research. Note also that many papers study multiple additional problem features and objectives, and thus can be found in different sections of the paper.

* Corresponding author.

E-mail addresses: dauzere-peres@emse.fr (S. Dauzère-Pérès), junwendong@hust.edu.cn (J. Ding), Liji.Shen@whu.edu (L. Shen), karim.tamssaouet@bi.no (K. Tamssaouet).

Section 2 introduces the notations and formalizes the problem. Section 3 covers the most studied criterion, i.e., the makespan, while Section 4 discusses alternative criteria and multi-criteria optimization. Then, Section 5 presents constraints that are considered in addition to the classical constraints, and Section 6 focuses on additional problem characteristics. Section 7 surveys the solution approaches. Finally, Section 8 provides some conclusions and what we believe are interesting research directions.

2. Problem description and notations

The classical FJSP can be formalized as follows: A set $\mathcal{J}$ of jobs, a set $\mathcal{R}$ of machines (or more generally unitary renewable resources), and a set $\mathcal{O}$ of operations are given. Each job j consists of a sequence of n_j consecutive operations in $\mathcal{O}$ (and thus $\sum_{j=1}^n n_j = |\mathcal{O}|$), and each operation $i \in \mathcal{O}$ can be processed on any machine among a subset $\mathcal{R}_i \subseteq \mathcal{R}$ of compatible (also called eligible) machines. The processing time of operation i on machine $k \in \mathcal{R}_i$ is given by p_i^k . According to the technological processing order of jobs, called routing or route, each operation i has one direct predecessor $pr(i)$ and one direct successor $fr(i)$ in the basic setting of the FJSP. Sets $\mathcal{PR}(i)$ and $\mathcal{FR}(i)$ contain respectively all predecessors and successors of i in its job routing. Furthermore, the problem is subject to the following assumptions:

1. All machines are available at time 0;
2. All jobs are available at time 0;
3. Preemption is not allowed, i.e., each operation must be completed without interruption once started;
4. Machines cannot perform more than one operation at a time.

A solution $\Pi = (a, \pi)$ consists of an assignment (a) , where $a(i)$ indicates the machine assigned to operation i , and a sequence π_k on each machine k ($\pi = \cup_k \pi_k$) that optimizes a given objective function. The most classical criterion is the minimization of the time required to complete all jobs, i.e., the makespan $C_{\max}$. In the resulting solution, $ps(i)$ and $fs(i)$ are the direct predecessor and direct successor of operation i in the sequence on machine $a(i)$, and sets $\mathcal{PS}(i)$ and $\mathcal{FS}(i)$ include all predecessors and successors of i on $a(i)$. Therefore, $\mathcal{P}(i) = \mathcal{PR}(i) \cup \mathcal{PS}(i)$ and $\mathcal{F}(i) = \mathcal{FR}(i) \cup \mathcal{FS}(i)$ contain all predecessors and successors of operation i , respectively.

Section 2.1 formalizes the FJSP as a Mixed Integer Linear Program, and Section 2.2 using a disjunctive graph model. Interested readers can refer to Kasapidis et al. (2021) and Naderi & Roshanaei (2022) for a constraint programming model. Section 2.3 extends the well-known three-field scheduling problem notation $\alpha|\beta|\gamma$ to encompass the different extensions of the FJSP discussed in the paper.

2.1. Mathematical modeling

To model the classical FJSP, we define the decision variables t_i as the start time of operation i , $C_{\max}$ the resulting makespan, as well as the binary variables below:

$$\alpha_i^k = \begin{cases} 1 & \text{if operation } i \text{ is assigned to machine } k, \\ 0 & \text{otherwise,} \end{cases}$$

$$\beta_{i' i} = \begin{cases} 1 & \text{if operation } i \text{ is sequenced before } i', \\ 0 & \text{otherwise.} \end{cases}$$

The Mixed Integer Linear Program (MILP) is presented below:

$$\min C_{\max} \quad (1)$$

$$\sum_{k \in \mathcal{R}_i} \alpha_i^k = 1 \quad \forall i \in \mathcal{O}, \quad (2)$$

$$t_i \geq t_{pr(i)} + \sum_{k \in \mathcal{R}_{pr(i)}} p_{pr(i)}^k \alpha_{pr(i)}^k \quad \forall i \in \mathcal{O}, \quad (3)$$

$$t_i \geq t_{i'} + p_{i'}^k - (2 - \alpha_i^k - \alpha_{i'}^k + \beta_{i' i})H, \\ \forall (i, i') \in \mathcal{O} \times \mathcal{O}, \quad \text{s.t. } i \neq i', \quad k \in \mathcal{R}_i \cap \mathcal{R}_{i'}, \quad (4)$$

$$t_{i'} \geq t_i + p_i^k - (3 - \alpha_i^k - \alpha_{i'}^k - \beta_{i' i})H, \\ \forall (i, i') \in \mathcal{O} \times \mathcal{O}, \quad \text{s.t. } i \neq i', \quad k \in \mathcal{R}_i \cap \mathcal{R}_{i'}, \quad (5)$$

$$C_{\max} \geq t_i + \sum_{k \in \mathcal{R}_i} p_i^k \alpha_i^k \quad \forall i \in \mathcal{O}, \quad (6)$$

$$\alpha_i^k \in \{0, 1\} \quad \forall i \in \mathcal{O}, k \in \mathcal{R}_i, \quad (7)$$

$$\beta_{i' i} \in \{0, 1\} \quad \forall (i, i') \in \mathcal{O} \times \mathcal{O} \quad (8)$$

The objective is to minimize the makespan $C_{\max}$. Constraints (2) ensure that each operation is assigned to one and only one of its eligible machines. Constraints (3) define the precedence relations between consecutive operations of the same job. Constraints (4) and (5) prevent the overlapping of two operations on the same machine k , where H is a sufficiently large value. These constraints are only relevant when operations i and i' are both assigned to machine k , i.e., $\alpha_i^k = \alpha_{i'}^k = 1$. Constraints (6) are used to determine the makespan.

2.2. Disjunctive graph modeling

A popular alternative to model the FJSP is to use a *disjunctive graph* $G = (\mathcal{N}, \mathcal{A}, \mathcal{E})$. Set $\mathcal{N}$ contains two fictitious nodes 0 and *, and each remaining node is associated with an operation. Hence,

$$\mathcal{N} = \{0, *\} \cup \mathcal{O},$$

which is decomposed into subsets containing operations of a single job.

Subset $\mathcal{N}_j$ models the n_j operations of job j , i.e.:

$$\mathcal{N}_j = \left\{ \sum_{j'=1}^{j-1} n_{j'} + 1, \dots, \sum_{j'=1}^{j-1} n_{j'} + n_j \right\},$$

where $\sum_{j'=1}^{j-1} n_{j'}$ is the number of operations of the first $j-1$ jobs.

Each job precedence relation is given by a conjunctive arc in set $\mathcal{A}$. The first and last operations of each job are connected to nodes 0 and *, respectively. Pairs of operations that can be assigned to the same machine k (i.e., operations i and v such that $k \in \mathcal{R}_i \cap \mathcal{R}_v$) are connected by disjunctive arcs in set $\mathcal{E}$. A disjunctive arc between nodes i and v can be defined either as a single arc pointing both to i and v , or as a pair of conjunctive arcs (i, v) and (v, i) . Note that there might be several disjunctive arcs between two operations i and v , one for each machine in $\mathcal{R}_i \cap \mathcal{R}_v$. Solving the FJSP means replacing each disjunctive arc by a conjunctive arc for the operations assigned to the same machine, and deleting all the other disjunctive arcs. Details on modeling with a disjunctive graph both the assignment and sequencing of operations in the FJSP can be found in Dauzère-Pérès & Paulli (1997). After the disjunctive graph is fully transformed into a conjunctive graph, the complete selection $\mathcal{S} = \cup_k \mathcal{S}_k$, where $\mathcal{S}_k$ includes the conjunctive arcs for operations assigned to machine k , is equivalent to the solution $\Pi = (a, \pi)$, as both the assignment and the sequencing of operations are decided in selection $\mathcal{S}$. Hence, $\mathcal{S}$ induces both the assignment variables α_i^k and the sequencing variables $\beta_{i' i}$ in the

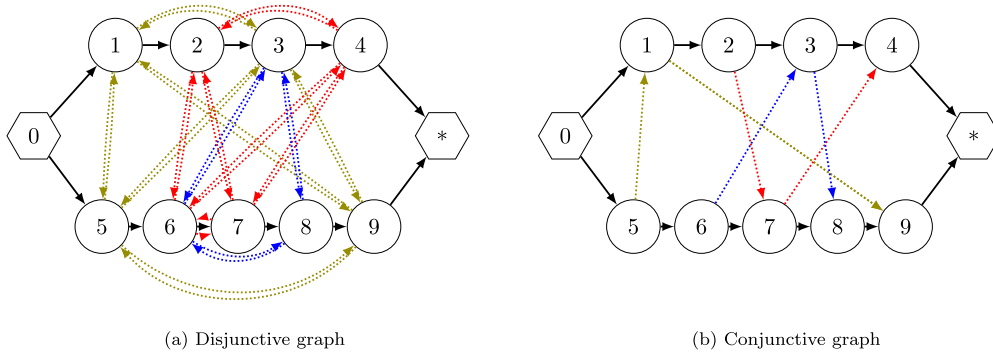


Fig. 1. Illustration of disjunctive graph.

MILP of Section 2.1. Note that $\mathcal{S}$ is feasible if and only if it induces an acyclic directed graph.

Next, we introduce some important definitions. A *path* from node i to node v in the resulting graph contains a finite sequence of nodes and conjunctive arcs between i and v . The *length* of a longest path connecting i and v , denoted by $L(i, v)$, is equal to the sum of the weights of the arcs on the path. Let l_{iu} be the *weight* of a conjunctive arc $(i, u) \in \mathcal{A} \cup \mathcal{E}$. For the classical FJSP, the weight can be defined as the processing time of the corresponding source node, i.e., $l_{iu} = p_i^{a(i)}$ for arcs (i, u) in both sets $\mathcal{A}$ and $\mathcal{E}$. Note that, although we consider weights on arcs, weights on nodes could also be considered. The weight of the arcs $(i, u) \in \mathcal{E}$ in the resulting directed graph can be modified and extended to accommodate further constraints in different FJSP settings.

A *critical path* refers to a longest path between the two fictitious nodes 0 and *, i.e., $L(0, *)$, and its length is the *makespan*. Operations and arcs on such paths are called *critical operations* and *critical arcs*, respectively. A *block* consists of a maximum sequence of adjacent critical operations that are processed on the same machine (Nowicki & Smutnicki, 1996). An operation is *internal* if it is neither the first nor the last operation of a block (Nowicki & Smutnicki, 1996).

For each operation i , we specify its *head* h_i and *tail* q_i . Graphically, h_i can be viewed as a longest path from 0 to i , whereas q_i indicates the time required to deliver all successors of operation i , i.e., the longest path from i to *:

$$h_i = \max_{u \in \mathcal{P}(i)} \{h_u + p_u^{a(u)}\}, \quad (9)$$

$$q_i = \max_{v \in \mathcal{F}(i)} \{p_v^{a(v)} + q_v\}. \quad (10)$$

In addition, it is assumed that the predecessor of the first operation in the routing of a job and the predecessor of the first operation on the sequence of a machine are the fictitious node 0, with $h_0 = p_0 = 0$. Similarly, the successor of the last operations in the routing of a job and the successor of the last operation on the sequence of a machine are the fictitious node *, with $q_* = p_* = 0$. The makespan is then determined by $C_{\max} = \max_{i \in \mathcal{O}} \{h_i + p_i^{a(i)} + q_i\}$.

In turn, a critical operation i satisfies the condition

$$h_i + p_i^{a(i)} + q_i = C_{\max}. \quad (11)$$

Figure 1 shows the disjunctive graph for a small FJSP instance with two jobs, 9 operations and three machines. Operations that can be processed on the same machine are connected with disjunctive arcs of the same color. It can be seen that, in this case, operations 3 and 6 have alternative processing machines.

2.3. Extended notations

To formally classify the different problem settings discussed in the paper, we use the α , β , and γ fields in the conventional three-field notation listed in Table 1. This notation is initially proposed in Graham et al. (1979) to characterize scheduling problems and their complexity, as a slight change in the features of a problem can lead to a shift in complexity class. Even if all versions of the FJSPs discussed in this paper are NP-hard, we believe this notation is still useful to quickly position the problem tackled in a paper without a long textual description. It is important for the notation to remain compact, but also to cover the problem features in practical settings and the scheduling literature. As the FJSP generalizes most of the academic machine scheduling problems, many features must be integrated in the notation. For example, overlapping and blocking constraints are not features of single or parallel machine scheduling problems.

We followed the above principles when choosing the notation in Table 1. We also try to adopt the most commonly used notation whenever possible. The associated sections in the paper are also provided in the table. As it can be observed, with a few exceptions, each field corresponds to one section: α in Section 6, β in Section 5 and γ in Section 4.

- Solving an FJSP involves three decisions: machine assignment, operation sequencing, and operation timing. The new notations in α roughly correspond to extensions introducing a new type of decision. Using J as a base, several extensions can be described by aligning the different prefixes. For example, *FSFMRJ* refers to a problem where jobs may have non-linear routes where the sequencing of non-connected operations should be chosen and where operations require multiple resources (and not a single machine), each to be selected from a given subset of resources.
- Field β contains the explicit constraints of the problem. To avoid a long list for this field, we consider the notation from the problem model instead of the original problem to solve. For example, the notation does not provide values corresponding to transportation constraints because these features can be modeled using minimum time lags or sequence-dependent setup times.
- Field γ contains the criterion/criteria to be optimized. In addition to the classical criteria, we introduced some new notations to cover different non-regular criteria. For these criteria, the notation refers to general managerial objectives rather than particular mathematical expressions of those objectives.

3. Makespan minimization

As it was the primary objective over decades of FJSP research, we first discuss papers where the makespan is minimized.

Table 1
Three-field notation used in the paper.

Field	Notation	Definition	Related section
Basic notations	d_j	Due date of job j	
	w_j	Weight of job j	
	C_j	Completion time of job j	
	L_j, E_j, T_j, U_j	Lateness, earliness, tardiness and unit penalty of job j	
α	J	Job-shop scheduling problem	
	$F\Box$	A machine may be selected in a given set	Section 2
	$FS\Box$	Flexible sequencing	Section 6.1
	$FP\Box$	Flexible process planning	Section 6.1
	$D\Box$	Distributed	Section 6.1
	$MR\Box$	Multi-resource	Section 6.2
	$MM\Box$	Multi-mode	Section 6.2
β	r_j	Release date of job j	Section 5.1
	$d_{ij}^{kk'}$	Minimum ($d_{ij}^{kk'} \geq 0$) and maximum time lags ($d_{ij}^{kk'} < 0$)	Section 5.1
	nwt	No-wait constraints	Section 5.1
	h_{kl}^f	Fixed unavailability periods	Sections 5.2 and 5.1
	h_{kl}^w	Flexible unavailability periods to be scheduled within a time window	Sections 5.2 and 5.1
	rs	Resumable operations after interruption by unavailability periods	Section 5.2
	$p - batch$	Parallel batching	Section 5.3
	$compatible$	$p - batch$ with compatible families	Section 5.3
	$incompatible$	$p - batch$ with incompatible families	Section 5.3
	s	Setup times	Section 5.4
	b	Nonzero, finite buffer capacity between machines	Section 5.5
	$block$	Blocking constraints	Section 5.5
	$overlap$	Overlapping	Section 5.7
	rrc	Recirculation or reentrant flows	Section 5.7
	bom	Bills of materials	Section 6.1
γ	reg	Any regular criteria	Section 4.1
	C_{max}	Makespan	Section 4.1
	TWF	Total weighted flow time, i.e., $\sum_{j=1}^n w_j(C_j - r_j)$	Section 4.1
	L_{max}	Maximum lateness, i.e., $\max_{j=1}^n L_j$	Section 4.1
	T_{max}	Maximum tardiness, i.e., $\max_{j=1}^n T_j$	Section 4.1
	TWT	Total weighted tardiness, i.e., $\sum_{j=1}^n w_j T_j$	Section 4.1
	$WNTJ$	Weighted number of tardy jobs, i.e., $\sum_{j=1}^n w_j U_j$	Section 4.1
	$nonreg$	Any non-regular criteria	Section 4.2
	W_T	Total workload of machines	Section 4.2
	W_M	Maximum workload	Section 4.2
	E/T	Earliness and tardiness	Section 4.2
	JIT	General Just-In-Time considerations	Section 4.2
	QR	Quality robustness	Section 4.2
	SR	Schedule robustness	Section 4.2
	EC	Energy cost	Section 4.2
	CF	Carbon footprint	Section 4.2

Brucker & Schlie (1990) address the FJSP by introducing the Multi-Purpose Machine (MPM) job shop. Systems with two jobs can be solved optimally. Based on two-job relaxations, lower bounds are proposed in Jurisch (1995). Early work on the classical FJSP to minimize makespan can be found in Barnes & Chambers (1996); Brandimarte (1993); Hurink et al. (1994) and Dauzère-Pérès & Paulli (1997), where the application of tabu search is dominant and various sets of benchmark problem instances are established.

3.1. Benchmark problem instances

There are several sets of FJSP benchmark instances that are widely used in the literature: *BRdata* (Brandimarte, 1993), *HUdata* (Hurink et al., 1994), *BCdata* (Barnes & Chambers, 1996), *DPdata* (Dauzère-Pérès & Paulli, 1997), *DMUdata* (Demirkol et al., 1998), *KCdata* (Kacem et al., 2002a), *Fdata* (Fattahi et al., 2007), *DAdata* (Birgin et al., 2015), as well as *Ydata* (Birgin et al., 2015), leading to a total of 467 instances of different sizes and with different flexibility levels. Some of the benchmark instances are available in the Monaldo Mastrolilli's web page,¹ and in the Oleg Shylo's web

page.² The detailed information of the benchmarks is presented in Table 2, where Column *flexibility* shows the average number of candidate machines for each operation ($\sum_{i \in \mathcal{O}} |\mathcal{R}_i| / \sum_{j=1}^n n_j$). Note that we separate sets *HUdata/sdata*, *DMUdata*, *Ydata* and *DAdata*, which correspond to the specific FJSP settings described in Table 2. We include these benchmarks for reference since they are commonly adopted in previous studies on the FJSP. Currently, best known results for these problem instances are summarized in Section 1 of the supplementary materials.

According to the solution quality on the well-known benchmark problem instances, studies on the FJSP to minimize makespan can be classified into the two categories: Papers using most benchmark instances discussed in Section 3.2 and papers using a small selected set of benchmark instances discussed in Section 3.3.

Note that many papers have focused on or are considering the makespan criterion while taking account other criteria, additional constraints or other characteristics, as discussed in Sections 4, 5 and 6. However, only a limited number of papers studying extensions of the FJSP with makespan minimization conduct computational experiments on benchmark instances of the basic problem. We believe this should be done more often to show that an ap-

¹ <https://people.idsia.ch/~monaldo/fjsp.html>.

² <http://optimizer.com/DMU.php>.

Table 2
Descriptions of benchmark instances .

Set	Size	Jobs n	Machines m	Operations n_j	Processing time	Flexibility
<i>BRdata</i>	21	{10, 15}	{11, 18}	{10, 15}	{5, 100}	{1.07, 1.3}
<i>HUdata/sdata*</i>	66	{6, 30}	{4, 15}	{4, 15}	{10, 100}	{1}
<i>HUdata/edata</i>	66	{6, 30}	{4, 15}	{4, 15}	{10, 100}	{1, 1.15}
<i>HUdata/rdata</i>	66	{6, 30}	{4, 15}	{4, 15}	{10, 100}	{1, 2}
<i>HUdata/vdata</i>	66	{6, 30}	{4, 15}	{4, 15}	{10, 100}	{1, 7.5}
<i>BCdata</i>	10	{10, 15, 20}	{4, 15}	{5, 15}	{1, 20}	{1.43, 4.1}
<i>DPdata</i>	18	{10, 15, 20}	{5, 8, 10}	{15, 25}	{10, 100}	{1.13, 5.02}
<i>DMUdata*</i>	80	{20, 50}	{15, 20}	{15, 20}	{10, 100}	{1}
<i>KCdata</i>	4	{4, 15}	{5, 10}	{5, 20}	{12, 56}	{1.0, 2.1}
<i>Fdata</i>	20	{2, 12}	{2, 8}	{2, 4}	{8, 30}	{1.21, 3.5}
<i>DAdata†</i>	30	{4, 12}	{5, 10}	{26, 127}	{1, 199}	{1.2, 3.67}
<i>Ydata†</i>	20	{4, 17}	{7, 26}	{24, 50}	{4, 38}	{1.3, 2.25}

* Instances for which each operation has exactly one candidate machine. † Instances with sequencing flexibility, where the precedence between the operations is given by an arbitrary directed acyclic graph.

proach solving a more general problem remains efficient for the classical FJSP.

3.2. Papers using most benchmark instances

Let us first consider studies where the proposed solution methods are thoroughly tested on the benchmark instances, resulting in new best-known upper bounds.

Following the integrated approach of [Dauzère-Pérès & Paulli \(1997\)](#); [Mastrolilli & Gambardella \(2000\)](#) introduce a Tabu Search (TS) with two neighborhood functions. An extensive computational study is conducted on the *BRdata*, *BCdata*, *DPdata*, and *HUdata* benchmark instances, and competitive baseline results are established. [Pezzella et al. \(2008\)](#) propose a Genetic Algorithm (GA), and experiments on the *BRdata*, *BCdata*, *DPdata*, and *HUdata* benchmark instances show that the GA obtains comparable results with respect to other best known algorithms. A two double-level parallel metaheuristic algorithm based on a new method for neighborhood determination is developed in [Bozejko et al. \(2010\)](#). Previous best known solutions were determined for the *BRdata*, *BCdata*, and *HUdata* benchmark instances with the aid of graphics processing units. [Hmida et al. \(2010\)](#) develop a variant of the climbing discrepancy search approach with various neighborhood structures, whose computational results on the *BRdata*, *BCdata*, *DPdata*, and *HUdata* benchmark instances show that the proposed approach outperforms previous algorithms on some types of benchmark instances and is competitive on the others. [Zhang et al. \(2011\)](#) propose a GA, whose computational results on the *BCdata*, *BRdata*, and *DPdata* benchmark instances show the same level or even better results in terms of computational times and quality compared to other GAs. [Yuan & Xu \(2013a\)](#) develop a hybrid differential evolution algorithm combined with a local search algorithm. Extensive computational experiments on the *KCdata*, *BRdata*, *BCdata*, and *HUdata* benchmark instances are convincing, and new best known solutions for some benchmark instances are determined.

[González et al. \(2015\)](#) develop a scatter search algorithm which determines new upper bounds for a number of instances in the *BRdata*, *BCdata*, *DPdata*, and *HUdata* benchmark instances. A GA hybridised with TS and heuristic seeding is presented in [Palacios et al. \(2015\)](#), leading to some new best known solutions for the *BCdata*, *BRdata*, and *DPdata* benchmark instances. [Li & Gao \(2016\)](#) present an algorithm hybridizing a GA and a TS algorithm, which provides some new best known solutions for the *BRdata*, *BCdata*, *DPdata*, and *HUdata* benchmark instances. A multi-start multi-level evolutionary local search is proposed in [Kemmoé-Tchomté et al. \(2017\)](#), which proves to be effective on the *BRdata*, *BCdata*, *DPdata*, and *HUdata* benchmark instances. [Caldeira & Gnanavelbabu \(2019\)](#) develop a Jaya algorithm, whose performance is well demonstrated on the *KCdata*, *BCdata*, *BRdata*, *DPdata*, *Fdata*,

and *HUdata* benchmark instances. [Ding et al. \(2019\)](#) present a two individual based evolutionary algorithm (MAE) which integrates a TS procedure. Computational results on the *BRdata*, *BCdata*, *DPdata*, and *HUdata* benchmark instances are convincing, and best known solutions are obtained for 10 hard instances.

3.3. Papers using a small selected set of benchmark instances

We now discuss papers focusing on specific benchmark sets.

[Wang et al. \(2012b\)](#) propose an Artificial Bee Colony algorithm (ABC), which is evaluated on the *KCdata* and *BRdata* benchmarks. A bi-population based estimation of distribution algorithm (BEDA) is presented in [Wang et al. \(2012a\)](#) which, in the experiments conducted on the *KCdata* and *BRdata* benchmark instances, outperforms some existing algorithms. [Jiang & Zhang \(2018\)](#) propose a population-oriented algorithm known as Grey Wolf Optimization, which is tested on the *KCdata* and *BRdata* benchmark instances.

[Chen et al. \(2020\)](#) develop a self-learning genetic algorithm (SLGA), whose numerical results on the *KCdata* and *BRdata* benchmark instances show that the SLGA significantly outperforms several other approaches. A particle swarm optimization algorithm (PSO) is proposed in [Ding & Gu \(2020\)](#), and is tested on the *BRdata* benchmark instances. [Denkena et al. \(2021\)](#) present quantum algorithms for the parallel FJSP, and conduct computational experiments on the *BRdata* benchmark instances, as well on a realistic use case. [Cao et al. \(2021\)](#) propose a knowledge-based cuckoo search algorithm, and conduct computational experiments on the *Ydata* and *DAdata* benchmark instances. [Liu et al. \(2021\)](#) develop a metaheuristic hybridizing GA and PSO, whose numerical results on the *KCdata*, *BRdata* and *Fdata* benchmark instances are competitive compared to previous algorithms. [Caldeira & Gnanavelbabu \(2021\)](#) present a Jaya algorithm with a local search procedure, whose effectiveness is illustrated on the *KCdata* benchmark instances.

4. Alternative criteria

As discussed in the previous section, the literature on the FJSP has initially mainly focused on the makespan criterion, as for most shop scheduling problems. More recently, additional criteria have been considered, primarily regular criteria as discussed in [Section 4.1](#). A regular criterion is a non-decreasing function of the completion times of the jobs. More precisely, when a regular criterion is optimized, it is always optimal to start the jobs as early as possible. This implies that the assignment and sequence of jobs on the machines corresponds to a single “left-shifted” schedule, and the optimal schedule is in the set of active schedules (see [French, 1982](#)).

Section 4.2 presents some of the challenges and the research conducted on non-regular criteria, which usually leads to a significant increase of the complexity as the schedule of operations should also be optimized in addition to their assignment and sequence on the machines. Finally, Section 4.3 shows that the optimization of multiple criteria is increasingly being considered in the literature.

4.1. Regular criteria

To solve the FJSP, operations must be sequenced on machines once they are assigned. However, a complete feasible selection provides only the processing sequence of operations through machines. As most optimization criteria in scheduling require the start times of operations, optimal operation start/completion times must also be optimized for the processing sequence. The process of deriving a schedule from a sequence is usually referred to as *timetabling* or *timing*. As for any optimization problem, the suitable approach to solving the timing problem strongly depends on the optimized objective function and its mathematical properties. Probably the most well-known class of scheduling criteria is the class of regular criteria. A criterion is said to be regular if it is a non-decreasing function of the completion times $C_1, C_2, \dots, C_n$ of the jobs. To refer to any regular criteria, $\gamma = \text{reg}$ is used in Table 1. To minimize a regular criterion while satisfying the constraints and the sequencing decisions, it is “only” necessary to consider semi-active schedules (see French, 1982), ensuring that operations are started as early as possible while satisfying the problem constraints and the fixed operation processing sequences. In other words, the timing problem can be modeled and solved as the longest path problem. By relaxing the requirements on satisfying the sequencing decisions, the search can be restricted to the set of active schedules, which is a subset of semi-active schedules. An active schedule is a schedule where no operation can be completed earlier by altering the processing sequence on the machines and not delaying other operations.

Except for the makespan, other common regular criteria in the scheduling literature are significantly less studied. Most regular criteria are based on the completion time of the job. A general criterion related to the cycle times of jobs and their priorities is the total weighted flow time $TWF = \sum_{j \in \mathcal{J}} w_j(C_j - r_j)$. The number of papers optimizing cycle time-related criteria in the context of FJSP is relatively small (e.g., Doh et al., 2013; Knopp et al., 2017; Lee et al., 2012; Thörnblad et al., 2015).

Criteria that use the due dates and completion times of jobs rely on the lateness L_j of job j , its tardiness T_j or its unit penalty U_j (see definitions in Table 1). The quality of a schedule can be evaluated by minimizing the weighted sum or max functions of L_j , T_j or U_j . To promote the completion of jobs as early as possible but not at the expense of others being extremely delayed, the maximum lateness $L_{\max} = \max_{j \in \mathcal{J}} L_j$ (Sourirajan & Uzsoy, 2007; Vilcot & Billaut, 2008) is relevant. When extreme delivery delays must be avoided, the maximum tardiness $T_{\max} = \max_{j \in \mathcal{J}} T_j$ is the most appropriate criterion (Doh et al., 2013; Loukil et al., 2007). The total weighted tardiness $TWT = \sum_{j \in \mathcal{J}} w_j T_j$ is relevant when delayed deliveries induce penalties that depend on the job (customer) and the duration of the delay (Mason et al., 2002; Poppenborg et al., 2012; Türkyılmaz et al., 2022). Finally, when the penalties do not depend on the duration of the delay, the weighted number of tardy jobs $WNTJ = \sum_{j \in \mathcal{J}} w_j U_j$ should be used (Baykasoglu, 2002; Doh et al., 2013).

By focusing only on the job completion times, the criteria discussed above fall short of capturing how the individual operations are performed. Semiconductor manufacturing is a typical application where job-based criteria are not the most relevant ones. As the cycle times of products in this context are very long (usu-

ally more than six weeks), the due dates of the complete jobs are often not relevant to solve a scheduling problem over, at most, a few days. Also, when solving a deterministic scheduling problem in a dynamic environment on a rolling horizon, minimizing the makespan is irrelevant to ensure a high level of resource utilization. To maximize the throughput in such a context, Yugma et al. (2012) introduce a new criterion that can be approximately modeled as the weighted number of tardy operations: Each operation is associated with a weight, and the end of the scheduling horizon is a common due date for all operations. Minimizing the weighted number of tardy operations leads to the maximization of the throughput. Ensuring high throughput is not necessarily the best objective to achieve. For example, semiconductor facilities define production targets for each product and process stage to ensure on-time delivery and linear flows in the factories. Tamssaouet et al. (2022) and Wu et al. (2022) introduce new criteria, based on the operation completion times and the end of the scheduling horizon, that model the satisfaction of production targets imposed by the master production plan. These are only a few examples showing the interest in optimizing criteria based on the operation completion times and not only the final operations of jobs in their routes. Such interest will be more apparent in the following section discussing non-regular criteria. Note also that the definition of regular criteria given above should be generalized to consider operations completion times instead of only job completion times.

4.2. Non-regular criteria

While most of the literature deals with regular criteria, more and more studies are optimizing non-regular criteria ($\gamma = \text{nonreg}$ in Table 1). However, no optimal schedule might be semi-active when non-regular criteria are considered. Therefore, timing decisions must be explicitly handled besides assignment and sequencing decisions in solution approaches. Below, we review criteria that need to be optimized for machine workload, just-in-time, robustness, energy, and carbon footprint considerations.

After the makespan, workload-based criteria are the most optimized criteria in the FJSP literature, as already noted in Chaudhry & Khan (2016), usually in a multi-criteria context (Türkyılmaz et al., 2020). More specifically, the total workload ($\gamma = W_T$) is the total working time assigned over all machines, and the maximum workload ($\gamma = W_M$) is the maximum working time of any machine. The total workload is relevant when machines are unrelated, i.e., the processing time of an operation differs from machine to machine. The total operating cost is a generalization of the total workload, where the cost of assigning an operation to a machine is not simply the processing time (Frutos et al., 2010). As a min-max criterion, the maximum workload aims to ensure the workload balance between machines by preventing excessive workload to be assigned to a single machine. Instead of the min-max criterion, other measures of “fairness” are considered, e.g., the variance between the working time of each machine with respect to the average processing time of all machines in Bezoui et al. (2023).

The workload-based criteria are not relevant in the case of shop scheduling problems with no machine flexibility, as they only evaluate the quality of the assignment decisions of operations to machines. Contrary to most other criteria, sequencing or timing decisions do not impact workload-based criteria. This explains why multiple objectives are optimized in the literature and not only a single workload-based criterion. Also, the question of what is the smallest class of schedules that includes optimal schedules is not relevant. Therefore, one can categorize such criteria in the class of regular criteria as they are non-decreasing in the completion times of jobs.

For most non-regular criteria, the set of active schedules or even semi-active schedules may not include an optimal schedule.

It might be optimal to delay the start time of an operation or to insert unnecessary idle times on a machine. For example, the earliness of job j , $E_j = \max\{d_j - C_j, 0\}$, is used to consider the various costs associated with completing a job too early in a just-in-time (JIT) environment. While the total weighted tardiness is a regular criterion, the total weighted earliness $\sum_{j \in \mathcal{J}} w_j^E E_j$ is called *antiregular criterion* (Brucker & Knust, 2012), for which a latest-start schedule is optimal. However, the sum of the weighted earliness and weighted tardiness $\sum_{j \in \mathcal{J}} w_j^E E_j + \sum_{j \in \mathcal{J}} w_j^T T_j$ is neither a regular nor an antiregular criterion. Instead of linear combination, it is also common to use the quadratic earliness and tardiness penalties. The different mathematical expressions related to earliness and tardiness are referred to as $\gamma = E/T$ in Table 1.

Compared to other scheduling problems, JIT criteria have rarely been studied in the FJSP. The makespan and the mean of earliness and tardiness are considered in Gao et al. (2014), while Gao et al. (2015) also optimize workload-based criteria for the FJSP with new job insertion. A common due date d for all jobs is a common assumption in the scheduling literature, as in Zambrano Rey et al. (2014) and Zambrano Rey et al. (2015) for the FSJP, where quadratic earliness and tardiness costs are minimized to avoid large deviations from d . Some papers associated a due window $[d_j^l, d_j^r]$ instead of a single due date d_j so that tardiness or earliness penalties are not incurred when the job is completed within the time window. The sum of earliness and tardiness costs is minimized in Huang et al. (2013), while additional constraints (sequence-dependent setup times and transportation times) are added in Sun et al. (2021) to minimize the makespan, the total workload, the workload of the critical machine, and earliness/tardiness penalties. Considering a case study from a furniture manufacturing company producing customized goods, Mohammadi et al. (2020) propose a bi-objective MILP to solve an integrated production flexible job shop scheduling and vehicle routing problem with a time window. The sum of the production and distribution scheduling costs is first minimized, and then the weighted sum of delivery earliness and tardiness is minimized.

Instead of only considering the completion times of jobs, some research is also interested in the completion times of operations as in Gomes et al. (2005) and Gomes et al. (2013), notably to minimize the queue of waiting jobs in front of machines. This is particularly important when machines are flexible as in the FJSP. Gomes et al. (2013) propose an MILP model for an FJSP in the mould-making industry with job recirculation and a final assembly stage. As this is a make-to-order environment, the weighted sum of earliness and tardiness for the set of orders and the intermediate storage time for the set of components are minimized. In Table 1, $\gamma = JIT$ is used to refer to non-regular criteria reflecting the objective of reducing inventory levels of either in-process or finished products. E/T criteria are part of *JIT* ones.

Non-regular criteria also arise when scheduling in the presence of real-time events (machine failures, urgent jobs, ...), termed dynamic scheduling. In the literature (see e.g., Aytug et al., 2005), dynamic scheduling approaches are grouped into three categories: Completely reactive, predictive-reactive, and robust pro-active. In addition to traditional optimization criteria, schedules' robustness and stability should also be considered. For the case of predictive-reactive scheduling, rescheduling can be performed to minimize the deviation of the new schedule quality or the operations start times from the original ones. Fattahi & Fallahi (2010) and Shen & Yao (2015) are examples of studies that consider stability measures in rescheduling decisions. The arrival of new jobs is considered in Fattahi & Fallahi (2010) when optimizing the makespan and the schedule stability. Shen & Yao (2015) consider machine breakdowns in addition and optimize the makespan, the total weighted tardiness, the maximum workload, and a stability measure. The

stability measure includes three terms related to the operations brought forward, delayed operations, and operations with delayed completion times.

Robust pro-active scheduling approaches focus on minimizing the impact of disruptions, usually by optimizing some robustness measures related to quality or schedule stability. Uncertainty, which is also discussed in Section 6.3, can be modeled as a set of scenarios associated with different possible realizations. Al-Hinai & Elmekawy (2011) and Ahmadi et al. (2016) consider machine breakdowns when determining the predictive schedule, and optimize the makespan and a stability measure. As stability measures can be computationally expensive, it might be more relevant to use surrogate robustness measures based, for example, on operation slacks and machine workload. This is the case in Xiong et al. (2013) for the FJSP with random machine breakdowns. Whether using the predictive-reactive or the robust approach, $\gamma = QR$ and $\gamma = SR$ refer to quality robustness and schedule robustness (stability) criteria, respectively.

Climate change challenges push companies to significantly reduce their energy consumption and carbon footprint ($\gamma \in \{EC, CF\}$). Regarding energy, several reduction measures can be adopted when scheduling the production: Reducing machine idle time, adjusting machine speed, or producing during the off-peak periods. A solution to reduce the idle time is to turn off the machines when knowing they will be kept idle for a certain time. The assignment decisions in the FJSP can impact the overall energy consumption, contrary to the JSP. Wu & Sun (2018) minimize the number of turning-on/off machines as they harm the machine performance and age while minimizing the total energy consumption. By considering electricity pricing, Park & Ham (2022) propose to minimize the total energy cost to shift production from peak to non-peak periods. Mokhtari & Hasani (2017) minimize the energy cost incurred by production and maintenance, and the cost incurred when the consumption of two previous activities exceeds a critical peak. Instead of energy consumption or cost, Piroozfard et al. (2018) minimize the total emitted carbon footprint during busy and idle times of the machines.

When optimizing the different criteria presented in this section, semi-active schedules might not be optimal. For example, delaying operations might help to reduce energy costs by producing in non-peak periods. Except for MILP models, timing decisions are usually not explicitly modeled in the heuristic approaches of the literature. Zambrano Rey et al. (2014, 2015) and Gao et al. (2014) are among the few examples proposing approaches that consider the non-regular aspect of E/T criteria. Zambrano Rey et al. (2014) and Zambrano Rey et al. (2015) develop approaches that consider the release date of jobs as decision variables. Gao et al. (2014) uses a neighborhood structure within a hybrid approach that is adapted to E/T criteria. However, the decoding method used in Gao et al. (2014) produces active schedules. Even if semi-active schedules might not be optimal regarding non-regular criteria, considering them instead of the subset of active schedules would lead to better schedules regarding E/T criteria. Shen et al. (2023) address an FJSP where the total energy cost is minimized with Time-of-Use tariffs and where a maximum makespan should not be violated. To determine non-active schedules, the authors propose two heuristic procedures which are also integrated in a metaheuristic framework.

4.3. Multi-criteria

Many real-world scheduling problems are multi-objective in nature. Multi-objective optimization problems consider several, conflicting or not, criteria simultaneously. A general overview of multi-objective optimization for scheduling problems is given in T'kindt & Billaut (2006). Surveys covering the solution ap-

proaches for the multi-objective FJSP can be found in Chaudhry & Khan (2016) and Türkyılmaz et al. (2020). This section aims at discussing the main questions that should be dealt with when optimizing multiple criteria, and at highlighting some of the important and recent solution approaches for the multi-objective FJSP or its extensions. This section is structured according to the categories defined by the occasion, relatively to the search process, when decision-maker preferences are used as the basis of tie-breaking between solutions in the efficient set: a priori (before) and a posteriori (after). To our best knowledge, interactive approaches solving the FJSP are not available in the literature.

As it is probably the most straightforward approach to solve a multi-objective optimization problem, the weighted sum of the criteria has been used in the earliest a priori approaches for the FJSP. This is, for instance, the case in Xia & Wu (2005) for the makespan, the total workload and the maximum workload. The same objectives are considered in a lexicographic order in Gao et al. (2008). More recently, the same problem is solved by Bissoli et al. (2021) with a clustering search algorithm. Park & Ham (2022) consider a lexicographic order to optimize the makespan and the total energy cost. Motivated by a practical scheduling problem, Tamssaouet et al. (2022) introduce a preference model where the trade-off is only allowed between some criteria, using simultaneously a lexicographic order and aggregation functions (weighted sum or weighted Tchebychev metric). Zhang & Yang (2016) solve the FJSP with flexible workdays, preemption, and overlapping operations to maximize a satisfaction criterion defined through goal programming. Given an aspiration level and the maximum acceptable level, the satisfaction criterion considers job delays, machine idle times, and machine overtimes. Five multi-objective decision-making methods, namely weighted sum, goal programming, goal attainment, LP metric, and max-min, are used in Ceylan et al. (2021) to solve a multi-stage supply chain problem where the production stage is modeled as an FJSP, and the total cost and the makespan are minimized.

In some approaches, Pareto dominance is used to decide whether a solution should be stored in an archive that approximates the Pareto front after termination. For example, a priori approaches can be augmented with an archive that stores the set of non-dominated solutions in addition to the best solution for the decision maker preferences (e.g., Tamssaouet et al., 2022). Some proper a posteriori approaches, except for the absence of the preferences, can be rather similar to a priori approaches. Such approaches can for example rely on the weighted sum but with weights varying during the search. Kacem et al. (2002b) use fuzzy logic to dynamically update the weights to control the search direction by prioritizing the optimization of the objective functions for which the mean of values in the population is far or not from the lower bound. Loukil et al. (2007) use a predefined set of diversified weight vectors to properly approximate the Pareto front while optimizing a set of regular criteria.

Another method that relies on the dominance relation only when updating the archive solves a single-objective problem at each iteration. The two-phase approach of Kacem et al. (2002a) is among the first to deal with the FJSP while optimizing the makespan and the total machine workload. Instead of alternating the criterion to optimize, more sophisticated approaches are explored in García-León et al. (2019), where a Pareto-based local search approach that can handle any combination of regular criteria is proposed. Moreover, ϵ -constraint approaches, where a single criterion is optimized while the others are bounded, are also popular. To solve an integrated flexible job-shop-vehicle routing problem with a time window, Mohammadi et al. (2020) use an ϵ -constraint approach to solve an MILP where two criteria are minimized: The sum of the production and distribution scheduling costs and the weighted sum of delivery earliness and tardiness. In

Vilcot & Billaut (2011), to find a set of non-dominated solutions when minimizing the makespan and the maximum lateness, the weighted sum with varying weights and an ϵ -constraint approach are compared.

Instead of handling the optimization criteria separately during the search or through aggregation functions, Pareto dominance can be directly used to approximate the Pareto front. This is usually the case with multi-objective evolutionary algorithms (e.g., NSGA-II and NPGA). Such an approach is used in Ahmadi et al. (2016) to optimize the makespan and a stability measure. To determine the Pareto front of small and medium instances, Soto et al. (2020) present a parallel Branch and Bound approach (B&B) to solve the FJSP where the makespan, the total workload, and the maximum workload are minimized. The rank-based fitness can be enriched by other measures that can help to increase the convergence or the diversity of the non-dominated solution set produced by evolutionary algorithms. For example, the approach in Türkyılmaz et al. (2022) relies on the Pareto ranking and the hypervolume contribution measure to solve an FJSP where both the makespan and the total tardiness are minimized.

5. Further constraints

The classical FJSP is an academic optimization problem. To be able to model and solve practical scheduling problems, the FJSP should be extended to encompass additional constraints. This section presents the constraints that are the most studied in the literature or the most relevant in practice.

5.1. Time lag constraints

There are many situations in practice where the waiting time of a job between two operations on its route is bounded from below or above for different considerations. The extreme case, commonly known as no-wait ($\beta = nwt$), is when no waiting time is allowed, i.e., the operations of jobs must be carried out without interruptions. For example, dealing with hot liquid glass imposes no-wait constraints between melting the glass and shaping it in glass manufacturing (Alvarez-Valdes et al., 2005). In many cases, jobs are allowed to wait at most a given maximum time, as for example in semiconductor manufacturing to prevent product contamination and jobs from being scrapped or reworked (Mönch et al., 2011). A minimum waiting time between two operations can also be imposed, to model for instance transportation delays between two machines or processing times of operations that do not require resources (such as drying or cooling down) or require non-bottleneck machines (Tamssaouet et al., 2022).

These practical considerations can be captured by introducing minimum or maximum time lags ($\beta = d_{ii'}^{kk'}$). Let us consider the case where such constraints are related to two consecutive operations i and $i' = fr(i)$ of a job processed respectively on machines k and k' . The time lags define a time window of i' with respect to i with a length $d_{ii'}^{kk'}$ which is not necessarily positive. For simplicity, we assume here that these constraints bind the start times of the operations. Let $\mathcal{L}$ be the set of all time lags to consider. Each time lag is denoted by $\lambda = (i, k, i', k', d_{ii'}^{kk'})$, where i and i' are operations in $\mathcal{O}$, $k \in \mathcal{R}_i$ and $k' \in \mathcal{R}_{i'}$ machines on which the two consecutive operations can be performed, and $d_{ii'}^{kk'}$ is the duration of the time lag. The mathematical model of the flexible job shop-scheduling problem can be extended by adding the following constraints:

$$t_i + d_{ii'}^{kk'} \leq t_{i'} + (2 - \alpha_i^k - \alpha_{i'}^{k'})H \quad \forall \lambda = (i, k, i', k', d_{ii'}^{kk'}) \in \mathcal{L} \quad (12)$$

Some remarks are discussed below.

- Other relations can be found in practice instead of start-to-start relations, such as completion-to-start or start-to-completion.

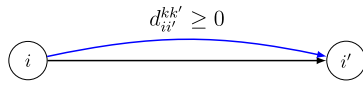


Fig. 2. Minimum time lags.

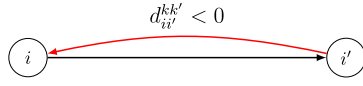


Fig. 3. Maximum time lags.

Note that, because the time lags are defined as dependent on the assigned machines, the relations can be transformed, for instance, to start-to-start relations by adding to or subtracting from the original time lag duration the processing times of one or two of the related operations.

- Time lags can occur for arbitrary pairs of operations of the same job. Thus, time lags can be adjacent or overlapping, and, at the same operation, a time lag can start, and another one can end. This is, for example, the case in semiconductor manufacturing where maximum time lags tend to follow each other in close succession and overlap (Lima et al., 2021).
- The extension of the disjunctive graph to include time lags is quite straightforward. As shown in Fig. 2, each minimum time lag (i.e., $d_{ii'}^{kk'} \geq 0$) induces an additional arc (i, i') with a weight $d_{ii'}^{kk'}$. As shown in Fig. 3, each maximum time lag (i.e., $d_{i'i}^{kk'} < 0$) induces an additional arc (i', i) with a weight $d_{i'i}^{kk'}$. As the time lags depend on the assigned machines, assigning a machine to an operation induces the relevant time lags to be selected. Note that a schedule is feasible when a complete selection induces a directed graph without a positive cycle in the presence of maximum time lags.

In general, minimum time lags do not make the problem more complex as they induce only a positive delay in the start times of operations. On the contrary, maximum time lags make scheduling problems more complex, which may explain the scarcity of research considering maximum time lags in FJSPs. Mascis & Pacciarelli (2002) show that, in the no-wait JSP and contrary to the classical JSP, a feasible schedule might not exist when starting from a partial schedule. More importantly, finding or proving that such a schedule does not exist is NP-complete. Consequently, it is unlikely that the ideas underlying the construction heuristics, the local search, or the evolutionary algorithms solving the FJSP are sufficient to produce feasible and good quality solutions when considering maximum time lags.

Although they are relevant in practice, general time lags have rarely been considered in the FJSP literature. Most of the papers are application specific and there is a lack of general approaches. Raaymakers & Hoogeveen (2000) model a problem in the pharmaceutical industry as an FJSP with no-wait constraints and overlapping operations. Because of the no-wait constraints, the authors propose to move an entire job in the neighborhood instead of a single operation. Furthermore, an additional procedure is required to make the schedule feasible and left-justified after each move. Alvarez-Valdes et al. (2005) describe the design and implementation of a flexible job-shop scheduling system in a glass factory with no-wait constraints and several additional characteristics (multi-task machines, overlapping, availability constraints, precedence constraints between jobs). A non-regular objective function based on due dates is optimized. Sundar et al. (2013) propose an ABC to solve an FJSP with no-wait constraints and makespan minimization.

No-wait constraints are often associated with blocking constraints (see Section 5.5). For example, Liu & Kozan (2011) model

train scheduling problems as a no-wait blocking FJSP. Burdett & Kozan (2018) model the scheduling of health care activities in a hospital as an FJSP with makespan minimization and several additional constraints, such as no-wait, blocking, maximum time lags, setup and transfer times and unavailability periods. An exact column generation approach is proposed in Pei et al. (2020) by considering a simplified problem with only two stages and where all the operations of a job require the same processing time.

In Yugma et al. (2012), minimum and maximum time lags are used to model constraints in semiconductor manufacturing. As a disjunctive graph approach is used, a significant impact of maximum time lags is the high computational cost required to evaluate each solution. The same industrial context is studied in Wu et al. (2022). Also in semiconductor manufacturing, Tamssaouet et al. (2022) use minimum time lags to model transportation times between operations and non-bottleneck operations. Another practical setting that requires the use of time lags is found in seamless rolled ring manufacturing, as described in Boyer et al. (2021). The problem is extended by considering maximum time lags, sequence-dependent setup times, loading and unloading times and machine capacities.

5.2. Availability constraints

Machines are not continuously available in most practical settings due to breakdowns, product quality problems or preventive maintenance operations. In this section, the deterministic case is considered, whereas the stochastic case including e.g., breakdowns and repair processes are random, is discussed in Section 6.3.

When unavailability constraints are due to preventive maintenance operations, two cases can be differentiated: Fixed ($\beta = h_{kl}^f$) and flexible ($\beta = h_{kl}^w$). The first case where maintenance periods are fixed is the most studied in the literature. In the flexible case, maintenance periods also have decision variables, i.e., each maintenance period can be scheduled within a given time window. Let us associate to each machine k the set of r unavailability periods $\mathcal{H}_k = \{h_{k1}, \dots, h_{kl}, \dots, h_{kr}\}$. Each unavailability period h_{kl} is defined by its time window $[S_{kl}, E_{kl}]$ and its duration p_{kl} . The fixed case corresponds to an unavailability period satisfying $p_{kl} = E_{kl} - S_{kl}$. The flexible case corresponds to an unavailability period satisfying $p_{kl} < E_{kl} - S_{kl}$.

One way of integrating unavailability periods in the FJSP is to model each unavailability period h_{kl} as a job with one operation i to be processed on a set of eligible machines $\mathcal{R}_i = \{k\}$. Therefore, it is necessary to extend the definition of the set of operations $\mathcal{O}$ in the MILP of Section 2. More specifically, the set $\mathcal{O}$ in Constraints (4) and (5) should be extended to include the operations modeling the unavailability periods. However, this modification is not enough as it only ensures non-preemption. Time lags can be used to guarantee that unavailability periods are started and completed within their specified time windows. A minimum time lag of duration S_{kl} can be used to model the release date of a job modeling unavailability period h_{kl} . The unavailability period can be forced to be completed within the time window by adding a maximum time lag of duration $-(E_{kl} - p_{kl})$. These additional time lags link the operation modeling the unavailability periods and the dummy start operation. Then, as for example in Yugma et al. (2012), the MILP of Section 2 and the disjunctive graph model can be extended to model time lags associated to unavailability periods, whether fixed or flexible, as shown in Section 5.1.

Instead of explicitly modeling the unavailability periods in the disjunctive graph, they can be considered by adjusting the start dates of operations so that there is no overlap between the processing of operations and unavailability periods. This approach is used for example in Tamssaouet et al. (2022) to solve a problem

with fixed unavailability periods. This approach avoids resorting to computationally expensive label-correcting algorithms required because of the negative arc weights introduced by the maximum time lags. Fixed unavailability periods are also considered in Pham & Klinkert (2008), where availability periods are defined at the level of a mode. Zribi et al. (2008) is another example that extends the FJSP to include fixed unavailability periods. It is worth mentioning that the integration of unavailability periods invalidates some known properties. For example, it is shown in Tamssaouet et al. (2018) that the property of having a zero slack (Eq. (11)) is neither a necessary nor a sufficient condition for an operation to be critical.

Heuristics that handle flexible unavailability periods are proposed in Gao et al. (2006) and Rajkumar et al. (2010), the latter with multiple objectives. Instead of an MILP with sequencing variables, Thörnblad et al. (2015) propose a time-indexed MILP to model flexible unavailability periods and other additional features for a workshop producing aerospace components.

Interestingly, the papers assuming that operations can be interrupted by fixed unavailability periods are all motivated by practical complex scheduling problems, as for example in Alvarez-Valdes et al. (2005); Grobler et al. (2010); Lunardi et al. (2020) and Lunardi et al. (2021). In these papers, operations are *resumable* ($\beta = rs$), i.e., they can be resumed after being interrupted by an unavailability period.

5.3. Batching constraints

In classical shop scheduling problems, a machine can only process one job at a time. However, there are practical settings where operations are performed in batches. A *batch* is defined as a group of jobs that are processed jointly. A batch scheduling problem consists of grouping jobs into batches and scheduling these batches on machines. Two types of batching problems can be found: Parallel processing (*p*-batching) and sequential processing (*s*-batching) (Mönch et al., 2011). The term *p*-batching ($\beta = p - \text{batch}$) refers to the capability of machines to process more than one job at the same time. In this case, the processing time of the batch is given by the maximum processing time of the jobs in the batch. The term *s*-batching refers to the presence of sequence-dependent setup times. In this case, jobs are grouped into families so that major setup times only occur when starting the job of a new family, while there is no setup time or a minor setup time between two jobs of the same family. The remainder of this section is dedicated to *p*-batching and Section 5.4 discuss the integration of sequence-dependent setup times. Despite the numerous applications of parallel batching discussed in the literature such as aircraft manufacturing or healthcare, most of the reported literature deals with semiconductor manufacturing.

Two main cases are considered in the literature: Compatible ($\beta = p - \text{batch, compatible}$) and incompatible job families ($\beta = p - \text{batch, incompatible}$). In the first case (e.g., burn-in operations in semiconductor back-end manufacturing), all the jobs can be batched together as long as the fixed maximum batch size b_m of machine m is satisfied. With incompatible job families (e.g., diffusion operations in semiconductor front-end manufacturing), only jobs of the same family f can be batched together and the fixed maximum batch size b_{mf} of machine m might depend on family f . Batching machines are critical because they are often characterized by long processing times. There is often a trade-off between starting a non-full batch right away or delaying the start of the batch to complete it with one or more jobs arriving later.

Considering batching machines makes the problem more complex as batching decisions must be taken in addition to assignment and sequencing decisions. Given the significant changes that are

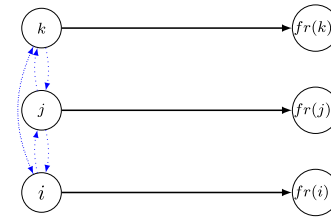


Fig. 4. Batching candidate operations.

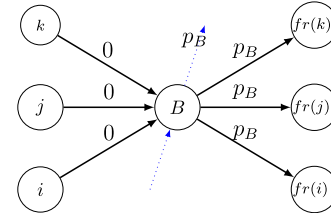


Fig. 5. Possible batching.

required to be made to the basic MILP of the FJSP to model batching machines, interested readers can find an MILP the inclusion of batch machines in the FJSP with compatible jobs in Ham & Cakici (2016) (as well as a constraint programming model), and with incompatible families in Mason et al. (2005).

The disjunctive graph can also be extended to integrate batching constraints. A widely used extension (e.g., Mason et al., 2002; Ovacik & Uzsoy, 2012) consists in introducing dedicated nodes to explicitly represent batching decisions. Figure 4 shows an example of three operations i , j and k that can be batched together. Assuming that the machine has a capacity of three jobs, i.e., $b = 3$, there are 7 ways of batching the jobs. Figure 5 shows when the three operations are part of batch B . A node B is added to model the start of the batch on the batching machine, and incoming and outgoing arcs with the right weights are also added.

Most of the studies propose heuristic approaches relying on disjunctive graph modeling. The most explored heuristics are variations of the Shifting Bottleneck (SB) heuristic of Adams et al. (1988). Mason et al. (2002) present a modified SB heuristic to solve $FJ|p - \text{batch, incompatible}, r_j, s_{jk}, recrc|TWT$. The experimental studies of Mason et al. (2002) and Mason et al. (2005) show that the proposed modified SB outperforms commonly used dispatching rules, and except for small instances, an MIP heuristic. To reduce the computational cost, Mönch & Drießel (2005) propose a two-layer hierarchical decomposition approach. Using a rolling horizon procedure, Sourirajan & Uzsoy (2007) also use a modified SB heuristic. Experimental results on large test instances generated show that the global scheduling policy combining temporal and workcenter-based decomposition gives significantly better performance than standard dispatching rules in reasonable CPU times. Mönch et al. (2007) investigate the impact of better sub-problem solution procedures, and the modified SB heuristic of Mason et al. (2002) is modified in Pfund et al. (2008) to consider multiple objectives. Drießel & Mönch (2012) consider automated material handling operations by decomposing into scheduling on production machines and scheduling on transportation vehicles.

Other approaches still relying on the disjunctive graph modeling are proposed in the most recent studies. Yugma et al. (2012) solve a flexible job-shop scheduling problem with many complex features and multiple practical objectives that are optimized using the weighted sum. Instead of inserting new batch nodes, operations in the same batch are linked with two inverse arcs with 0 weights to ensure the synchronization of the opera-

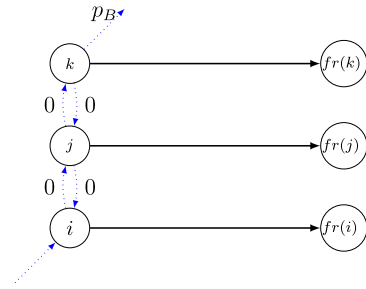


Fig. 6. Batch modeling in Yugma et al. (2012).

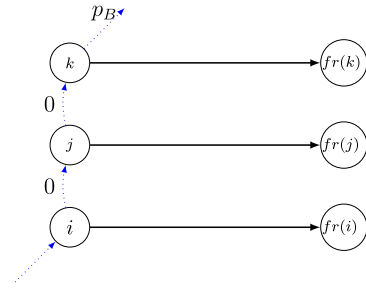


Fig. 7. Batch-oblivious conjunctive graph of Knopp et al. (2017).

tions included in the same batch. This modeling is illustrated in Fig. 6. Knopp et al. (2017) propose a novel extension, called batch-oblivious (Fig. 7), to avoid inserting dedicated nodes and arcs to explicitly represent batches, they are encoded in the arc weights. On the same complex scheduling problem, Wu et al. (2022) introduce properties to reduce the solution space that are used to develop an efficient algorithm that does not rely on a disjunctive graph model.

5.4. Setup time constraints

With respect to setup times (\$\beta = s\$), extensive research can be found in comprehensive surveys (e.g., Allahverdi, 2015), which summarize and classify a wide variety of setups raised in scheduling problems. Recent works on the FJSP with setups specifically address sequence-dependent setup times (SDSTs), denoted by \$s_{i' i}^k\$, where the setup length of the current operation \$i'\$ depends on its immediate predecessor \$i\$ on the same machine \$k\$. It is commonly assumed that the SDST is separable from the corresponding operation. In this case, SDSTs can be integrated in the mathematical model as follows:

$$t_i \geq t_{i'} + p_i^k + s_{i' i}^k - (2 - \alpha_i^k - \alpha_{i'}^k + \beta_{i' i})H, \quad \forall (i, i') \in \mathcal{O} \times \mathcal{O}, \text{ s.t. } i \neq i', k \in \mathcal{R}_i \cap \mathcal{R}_{i'}, \quad (13)$$

$$t_{i'} \geq t_i + p_{i'}^k + s_{i i'}^k - (3 - \alpha_i^k - \alpha_{i'}^k - \beta_{i i'})H, \quad \forall (i, i') \in \mathcal{O} \times \mathcal{O}, \text{ s.t. } i \neq i', k \in \mathcal{R}_i \cap \mathcal{R}_{i'}, \quad (14)$$

Regarding the disjunctive graph model, the weight of the disjunctive arcs should be increased by the associated SDSTs. The set \$\mathcal{E}\$ now also includes disjunctive arcs from node 0, indicating the potential first operation processed on this machine. Its weight is then equal to the initial setup time if available. Using the same example as in Fig. 1, the extended disjunctive arcs are illustrated in Fig. 8 for one machine (\$m_1\$). More specifically, a pair of SDSTs can be represented by two arcs of reverse directions as shown in Fig. 8 with arcs connecting nodes 1 and 9. Their weights are the sum of the corresponding processing times and setup times.

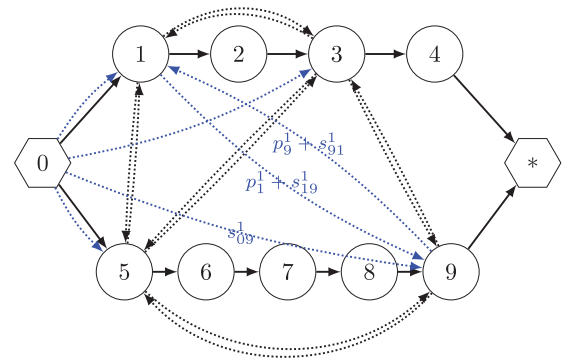


Fig. 8. Disjunctive graph with SDSTs.

Considering SDSTs, Mousakhani (2013) develop mathematical models and an iterated local search that minimize total tardiness. In Abdelmaguid (2015), SDSTs are assumed to be symmetric and small relative to processing times. To include SDSTs, the neighbourhood of Mastrolilli & Gambardella (2000) is extended, and randomized neighbourhoods are tested on modified benchmark instances. This approach achieves small deviations from lower bounds on problem instances. Shen et al. (2018) address the problem to minimize the makespan, and present a mathematical model and a tabu search algorithm with specific neighborhood functions and a diversification structure. Even when SDSTs are ignored, competitive results are obtained for benchmark instances.

SDSTs are also considered in extended FJSP settings. For assembly FJSP in dynamic environments, Zhang & Wang (2018) propose a Constraint Programming model and an MIP model, as well as dispatching rule based heuristics.

For the multi-objective problem, MIPs are proposed to minimize both makespan and total tardiness in Deliktas et al. (2019). Li et al. (2019) propose a hybrid algorithm with non-dominated sorting, which operates on a main population and an auxiliary population, to minimize both the makespan and the total setup cost. Li et al. (2020) present a Jaya algorithm combined with simulated annealing for the FJSP with SDSTs and transportation times to minimize a weighted objective of makespan and energy consumption.

Rossi & Dini (2007) stress separable setup times, also known as anticipatory or detached setups, which can be performed before the operation is available. For both sequence dependent and independent setup times as well as transportation times, Rossi & Dini (2007) present an ACO algorithm. For the problem with parallel machines and operation lag times, specific pheromone trail coding and ant colony operators are proposed. In Rossi (2014), the ACO is strengthened by a list scheduler and local search functions based on the disjunctive graph.

Setups that can only be carried out when operations are available are called inseparable, non-anticipatory or attached. If setup times are job-dependent, they can be included in the processing times and represented by the conjunctive arcs in set \$\mathcal{A}\$. In most cases where the length does not solely depend on jobs, an arc, whose weight and associated nodes depend on the case, can be added in the disjunctive graph to prevent a setup from being performed in advance.

For instance, attached setup times are assumed in González et al. (2015), where a scatter search algorithm is proposed, while Özgüven et al. (2012) include both attached and detached SDSTs with processing flexibility and present mathematical models. Moreover, when attached and detached SDSTs, machine release dates and time lag requirements are considered, a parallel GA is proposed to minimize the makespan in Defersha & Chen (2010), while Defersha & Rooyani (2020) develop a two-stage GA and conduct extensive computational experiments.

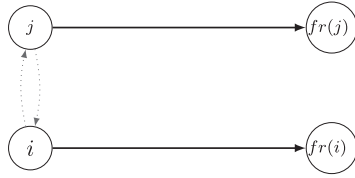


Fig. 9. Non-blocking operations.

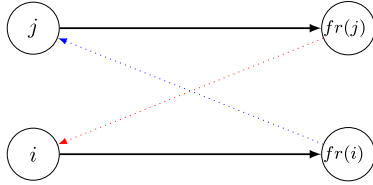


Fig. 10. Blocking operations.

5.5. Blocking constraints

Most shop scheduling problems assumed that there is enough buffer to store any number of jobs waiting to be processed on a machine. However, there are many practical situations where buffers have limited capacity. The extreme case is when there is no buffer, and a job is *blocked* on its current machine until its next machine is available. Railway scheduling problems are typical examples of scheduling problems with blocking constraints.

The mathematical model or the disjunctive graph presented in Section 2 are no longer valid. Let us consider the case where all the operations are blocking. Recall that $fr(i)$ refers to the route successor of operation i . Instead of Constraints (4) and (5), Constraints (15) and (16) are needed to solve the problem with blocking constraints.

$$t_i \geq t_{fr(i')} - (2 - \alpha_i^k - \alpha_{i'}^k + \beta_{i'})H, \quad \forall (i, i') \in \mathcal{O} \times \mathcal{O}, \text{ s.t. } i \neq i', k \in \mathcal{R}_i \cap \mathcal{R}_{i'}, \quad (15)$$

$$t_{i'} \geq t_{fr(i)} - (3 - \alpha_i^k - \alpha_{i'}^k - \beta_{i'})H, \quad \forall (i, i') \in \mathcal{O} \times \mathcal{O}, \text{ s.t. } i \neq i', k \in \mathcal{R}_i \cap \mathcal{R}_{i'}. \quad (16)$$

The adaptation of the disjunctive graph model to consider blocking constraints is known as *alternative graph* (Mascis & Pacciarelli, 2002) considering the JSP. Two operations i and i' that can be processed on the same machine k are connected by a pair of disjunctive arcs (i, i') and (i', i) in the disjunctive graph. An adaptation of the alternative graph in the case of FJSP is made by replacing the pair of disjunctive arcs (i, i') and (i', i) with a pair of alternative arcs $(fr(i), i')$ and $(fr(i'), i)$ with 0 as weights (Figs. 9 and 10). Similarly to the no-wait constraints, a complete selection is feasible if it induces a directed graph without a positive length cycle. To model situations where there are buffers with finite capacity, it is possible to consider all the storage positions as machines with zero processing time. The problem with limited buffers can therefore be studied as a problem with blocking constraints where all machines have no intermediate buffers.

Gomes et al. (2005) present ILP models to solve an FJSP in discrete part manufacturing industries with limited intermediate buffer and re-circulation. Pham & Klinkert (2008) model a surgical case scheduling problem as a multi-mode FJSP with blocking constraints and some additional features. The same problem is also considered in Mati & Xie (2011), who model a production workshop where key components of aircraft are produced with no buffer space to store products between machines. The study of scheduling problems with limited buffer capacity is also moti-

vated by the increasing importance of Automated Material Handling Systems (AMHS) in today's manufacturing systems. For example, Poppenborg et al. (2012) consider the problem of scheduling an FJSP with blocking constraints and additional features, where jobs are transported between the machines by Automated Guided Vehicles (AGVs).

Similarly to maximum time lags, finding feasible solutions is challenging for heuristics when blocking constraints are considered. Only feasible solutions are visited in the two-stage hybrid heuristic of Liu & Kozan (2011) and the TS of Poppenborg et al. (2012). Different neighborhood structures are defined to increase the chance of visiting a feasible neighbor. More sophisticated heuristic approaches are proposed in Burdett & Kozan (2010); Liu & Kozan (2011) and Burdett & Kozan (2018) by extending the disjunctive graph model. Other constructive approaches, such as the greedy heuristic proposed in Mati & Xie (2011), rely on procedures inserting jobs instead of single operations.

Among the most successful approaches to solve job-shop scheduling problems with blocking are those exploiting the theoretical results proposed in Gröflin & Klinkert (2006). As inserting one operation (in a construction heuristic or in a local search heuristic) may often lead to infeasible solutions, various studies rely on a job insertion based feasibility recovery (JIFR) procedure that converts an infeasible solution to a feasible one. Such procedure is used for example in Gröflin et al. (2011) to solve the problem with blocking constraints and transfer and setup times.

5.6. Transportation constraints

The integration of transportation constraints in the FJSP has been studied by various papers in the literature. The following two main cases can be differentiated.

- *Transportation times.* In this case, studied for example in Rossi (2014); Rossi & Dini (2007) and Karimi et al. (2017), transportation resources are assumed to be non bottleneck, and only the transportation times between the selected machines are modeled. Note that the flexibility of machines makes the problem difficult as reassigning an operation to another machine implies changing the travel time of the job from one machine to the next machine in the route. Travel times can actually be seen as minimum time lags $d_{i'k'}^{kk'}$ that depend on the machines k and k' to which consecutive operations i and i' on a route are assigned, see Section 5.1.
- *Transportation resources.* In this case, introduced by Deroussi & Norre (2010) and Kumar et al. (2011) (both references propose benchmark instances), transportation resources are explicitly considered and thus should also be assigned to the transportation operations, that also need to be explicitly modeled. The problem becomes even more complex than with transportation times as (1) Travel times of empty transportation resources to machines to pick up jobs should be modeled, and (2) Reassigning an operation to another machine implies changing the travel times of the transportation resource to and from the machine. An additional complexity is that transportation resources might induce blocking constraints, as discussed in Section 5.5.

We only consider the problem with transportation resources in the remainder of the section. Zhang et al. (2012a) propose a metaheuristic for the problem. The same authors extend this work in Zhang et al. (2014) to model input and output buffers of machines with limited space. They propose a modified SB heuristic by including transportation and storage nodes in the disjunctive graph. Deroussi (2014) and Nouri et al. (2016) propose metaheuristics, while Ham (2020) propose a Constraint Programming model to solve the instances of Deroussi & Norre (2010) and new benchmark instances adapted from Hurink et al. (1994).

Fontes et al. (2019) and Homayouni & Fontes (2021) propose two MILPs and a late-acceptance hill-climbing heuristic. This heuristic considers moves where one production operation and one transportation operation are reassigned and two production operations are swapped. Benchmark instances are also generated based on the FJSP instances from Fattahi et al. (2007) and Brandimarte (1993).

5.7. Other constraints

Other constraints have been studied, even if they can often be seen as special cases of those presented in the previous sections. To conclude Section 5, we discuss overlapping operations and reentrant flows below.

Overlapping operations In some practical settings, an operation may start before its predecessor in the job routing is completed. Such situations arise in the pharmaceutical industry (Raaymakers & Hoogeveen, 2000), the glass manufacturing (Alvarez-Valdes et al., 2005) and the printing industry (Lunardi et al., 2020). If a job is a lot of products, waiting for all products to be completed at an operation might not be necessary to transfer products and start the next operation. The minimum number of products that must be completed before products can be transferred from one machine to the next is called *transfer batch* (corresponding typically to the size of a container) in Ivens & Lambrecht (1996) and Schutten (1998). To ensure non-preemption and that operation $fr(i)$ is not completed before i , it is necessary to consider the processing time per product of each of the two overlapping operations and the size of the transfer batch. Ivens & Lambrecht (1996) and Schutten (1998) propose an extension of the disjunctive graph to model overlapping operations. MILP and CP models considering overlapping operations can be found in Lunardi et al. (2020). Examples of recent heuristic approaches can be found in Zhang & Yang (2016), Meng et al. (2018) and Lunardi et al. (2021).

Reentrant flows (recirculation) Most scheduling problems assume that a machine or a group of machines are only used once by a job in its route. This is not the case in some industrial contexts, the most well-known being probably semiconductor manufacturing which includes reentrant flows (see Mönnch et al. (2011)). More generally, because machines are flexible in the FJSP, the same machine can be assigned to two or more operations in the route of a job. This is the case of operations 5 and 9 in the conjunctive graph of Fig. 1. Hence, reentrant flows can actually occur in many benchmark instances. Heuristic approaches that explicitly consider reentrant flows in semiconductor manufacturing are proposed in Mason et al. (2002); Sourirajan & Uzsoy (2007) and Knopp et al. (2017). Other industrial contexts with recirculation can be found in the manufacturing of aerospace components in Thörnblad et al. (2015) and aircraft components in Mati et al. (2011). Gomes et al. (2013) propose an MILP for an FJSP with reentrant flows in the mold-making industry. Note that the disjunctive graph model does not require to be extended to take reentrant flows into account. Because of route precedence constraints, some disjunctive arcs can be deleted from the disjunctive graph. For example, as operation 5 precedes operation 9 on its job route, the disjunctive arc (9, 5) can already be deleted. It is also noteworthy to consider the case of operations 6 and 7 that can be processed on the same machine while being adjacent on the job route. In the JSP, such operations can be combined in one operation without missing any opportunity regarding the solution quality. However, with the flexibility in the FJSP, keeping such operations separate can lead to better solutions.

6. Additional characteristics

Again motivated by real-life contexts, other characteristics have been added to the FJSP. This section first covers in Section 6.1

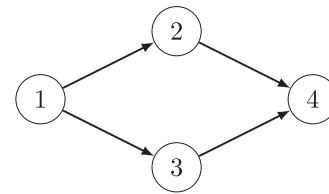


Fig. 11. Subgraph of an arbitrary non-linear route.

when jobs have complex routes, and in Section 6.2 when operations require multiple resources. Section 6.3 surveys studies that consider uncertainties, and Section 6.4 other characteristics not discussed in the previous sections.

6.1. Complex routes

The FJSP assumes that the route, i.e., the processing order of the operations, of a job is sequential. This is not always true in practice, where an operation may have more than one predecessor or more than one successor. An extension of the FJSP considers an arbitrary directed acyclic graph representing general precedence constraints between operations. The terminology to describe this extension is diverse and needs to be unified: Assembly and split structures in Ivens & Lambrecht (1996), non-linear routes in Dauzère-Pérès et al. (1998), convergent and divergent job routings in Schutten (1998), sequencing flexibility in Birgin et al. (2015) and arbitrary precedence constraints in Lunardi et al. (2020) and Kasapidis et al. (2021). In the remainder of this section, the term *non-linear routes* is used. To unify the previous studies, we also show how different situations can be modeled by non-linear routes. Let us illustrate the discussion with the precedence subgraph in Fig. 11, where Operation 1 has two successors, Operations 2 and 3, which are predecessors of Operation 4. Two cases can be distinguished based on whether Operations 2 and 3 are performed in parallel or not. The first case typically corresponds to when Operations 1 and 4 are disassembly and assembly operations, respectively. An important characteristic of this case is that the operations are not necessarily performed on the same physical item, i.e., 2 and 3 can be performed in parallel. Most of the problems studied in the literature fall into this case, such as Loukil et al. (2007); Vilcot & Billaut (2008) and Gomes et al. (2013) who consider assembly operations (multiple predecessors and at most one successor) arising in the brass products, mould making and the printing industries, respectively. Settings requiring arbitrary non-linear routes, i.e., an operation may have several predecessors and/or successors, are dealt with in Birgin et al. (2015); Dauzère-Pérès et al. (1998); Lunardi et al. (2021); Sobeyko & Mönnch (2016) and Kasapidis et al. (2021). MILP and CP models are proposed in Lunardi et al. (2020). Note that the disjunctive graph model is similar to that of the classical FJSP, except for the routes that are non-linear. As this extension does not involve other decisions than those of the basic FJSP, it is referred to as $\beta = bom$ (Bills of Materials).

Before presenting the second case, let us discuss the three types of manufacturing flexibility defined in Benjaafar & Ramakrishnan (1996). The first one, *operation flexibility*, corresponds to the FJSP where an operation can be performed on more than one machine. Some of the extensions discussed in this section integrate the two other types of flexibility: Sequencing flexibility and processing flexibility. The *sequencing flexibility* ($\alpha = FS$) is the capability of interchanging the order in which job operations are performed, i.e., when there are feasible alternate sequences of operations in a route. By this definition, the problem solved in Birgin et al. (2015) should not be named the FJSP with sequencing flexibility, as sequencing flexibility means that Operations 2 and 3 can

not be performed in parallel in Fig. 11. After completing 1, it is either possible to perform 2 then 3 (i.e., job has a linear route (1, 2, 3, 4), or the reverse sequence (i.e., job has a linear route (1, 3, 2, 4), but not in parallel. In addition to the decisions of assigning and sequencing operations, it is also necessary to sequence the job operations that are not linked with precedence constraints, similarly to the job operations in the Open Shop Problem. Contrary to the first case, only a few approaches can handle sequencing flexibility. One way of forbidding operations to overlap is to associate each job with a fictitious machine. Besides the actual machine, each operation would also require the fictitious machine of its job. Then, in addition to non-linear routes, operations should also require multiple resources. Dauzère-Pérès et al. (1998) and Kis (2003) are among the few articles proposing advanced heuristic approaches that tackle problems in the two cases.

The third type of flexibility is *processing flexibility* ($\alpha = FP\Box$), which is defined as the availability of alternative routes (also called process plans in the related literature) for each job. The most straightforward extension of FJSP to consider processing flexibility is to allow a job to have multiple linear routes ($\alpha = FPFJ$). Only one of the alternative routes should be selected in addition to assigning and sequencing operations. Özgüven et al. (2010), resp. Özgüven et al. (2012) study the FJSP with processing flexibility without, resp. with SDSTs. Doh et al. (2013) consider the problem where various parameters, such as job arrivals and processing times, are not deterministic.

Before introducing the most general extension regarding job routes, let us show that the FJSP with alternative routes is a generalization of another extension of the FJSP: The distributed FJSP ($\alpha = DFJ$). In the distributed FJSP, each job must be allocated to one manufacturing unit in a network of manufacturing units, each with a shop scheduling problem that can be modeled as the FJSP. Chan et al. (2006b); De Giovanni & Pezzella (2010), and Meng et al. (2020) are a few examples proposing exact and heuristic approaches to minimize the makespan in the context of the DFJSP. It can also be seen as a special case of $\alpha = FPFJ$, as the manufacturing units can be viewed as a set of alternative routes for each job. The main difference is that the sets of machines required by each route are disjoint in the $\alpha = DFJ$ extension, which is not necessarily the case in the $\alpha = FPFJ$ extension.

Considering the three types of flexibility, and all the extensions discussed above, it thus leads to the most general extension $\alpha = FPFJ$. In the resulting problem, instead of a DAG, the job routes are modeled using an And/Or graph. Such modeling allows operations to have multiple predecessors or successors through the And-subgraph. Through the Or-subgraph, operations can be succeeded by a set of parallel sequences of operations, from which only one should be selected. To the best of our knowledge, Kis (2003) is the only advanced approach that tackles this problem, as an And/Or graph for jobs is considered and operations require a fixed set of resources. The operation flexibility and processing flexibility can be modeled through the Or-subgraph. Lee et al. (2012) and Zhang et al. (2020) also use an And/Or graph to model complete routes. However, as the approaches assume that each operation requires only one machine, they do not tackle the sequencing flexibility discussed above. Zhang et al. (2020) consider a more general problem than Lee et al. (2012) by taking SDSTs and transportation times into account and optimizing the makespan, the total tardiness, and the total workload simultaneously.

6.2. Multiple resources

As in the majority of the literature on scheduling, most studies on the FJSP consider that each operation requires a single resource, typically a machine, to be processed. However, there are contexts where multiple primary resources or additional auxiliary

resources are necessary, e.g., a machine and an operator, two operators with different skills, or tools and fixtures. Scarce auxiliary resources must be considered to determine feasible and efficient schedules. Let us denote by $\alpha = FMRJ$ the extension of the FJSP where operations may require multiple necessary resources, and each necessary resource must be selected from a set of possible resources. Note that two sets of possible resources do not have to be disjoint for an operation, but the same resource cannot be assigned more than once as a necessary resource. The $\alpha = FMRJ$ extension is considered for the first time in Dauzère-Pérès et al. (1998) and recently in Kasapidis et al. (2023), and a practical context within a coal export terminal is reported in Burdett et al. (2020), which includes multiple resources and other extensions such as preemption. Note that, as shown in Dauzère-Pérès et al. (1998), an open shop scheduling problem (and thus a mixed shop scheduling problem, which combines jobs with fixed routes and jobs whose routes should be decided) can be modeled as a flexible job shop scheduling problem with multiple resources and $\alpha = FMRJ$. Consequently, given that the FJSP is a generalization of the JSP, $\alpha = FMRJ$ is also a generalization of the mixed shop scheduling problem.

The FJSP extension described above assumes that all combinations of necessary resources (one in each set of possible resources) are allowed. But this is not always the case in industry where some combinations of assignments may be forbidden, e.g., an operator might not be qualified to run some machines. The FJSP can be extended to consider operations requiring multiple resources by introducing the notion of *mode*, which corresponds to a subset of resources on which an operation can be processed. The multi-mode JSP, denoted $\alpha = MMJ$, can be seen as a generalization of $\alpha = FMRJ$ where a set of modes is associated with each operation. The set of modes is all the possible resource combinations in $\alpha = MMJ$. Differentiating $\alpha = FMRJ$ and $\alpha = MMJ$ is essential, not only from an application perspective but also from a methodological perspective. The $\alpha = FMRJ$ extension requires much less data to handle if the sets of possible resources are large, since one mode is created in the $\alpha = MMJ$ extension for every possible combination of necessary resources. Another important point is that it is easier to transfer the results from the FJSP to the $\alpha = FMRJ$ extension than to the $\alpha = MMJ$ extension. This is because, as pointed out in Dauzère-Pérès & Pavenau (2003), changing the assignment of an operation on a single necessary resource is possible in $\alpha = FMRJ$, whereas all the necessary resources should be considered when changing modes in the $\alpha = MMJ$ extension.

Brucker & Neyer (1998) is, to our best knowledge, the first to propose an advanced solution approach to solve the $\alpha = MMJ$ with makespan minimization. In addition to considering multiple modes, Kis (2003) proposes sophisticated methods to solve an extension of the FJSP with sequencing and processing plan flexibility. In this work, a unique mode is assigned to each operation, and flexibility is achieved through the Or-subgraphs. Dauzère-Pérès & Pavenau (2003) extends the work of Dauzère-Pérès et al. (1998) to handle the limitation of $\alpha = FMRJ$ compared to $\alpha = MMJ$ by preventing resources that cannot be combined (called forbidden sets of necessary resources) to be assigned. Another relevant extension proposed by Dauzère-Pérès & Pavenau (2003) aims at addressing the case where an assigned necessary resource can start or complete the operation before another assigned necessary resource. A surgical case scheduling problem is modeled in Pham & Klinkert (2008) as the $\alpha = MMJ$ extension with additional features such as blocking constraints, availability constraints, and setup times. Blocking constraints are also considered in the $\alpha = MMJ$ extension in Mati et al. (2011). Prot & Bellenguez-Morineau (2012) model a scheduling problem in the manufacturing of metal goods as the $\alpha = MMJ$ extension with SDSTs and some assembly operations.

Differently from the references discussed above, several works consider special cases where operations require at most two re-

sources. These problems are often referred to as dual-resource constrained scheduling problems. [Lei & Guo \(2014\)](#) solve the $\alpha = MMJ$ extension where each mode is a set of at most two resources. [Andrade-Pineda et al. \(2020\)](#) model the scheduling problem in an automobile collision repair shop as the $\alpha = FMRJ$ extension, where operations require at most two resources and there are some assembly/disassembly operations.

6.3. Uncertain environments

Most stochastic scheduling problems consider uncertain processing times or machine failures. Although the literature is relatively abundant on single machine, parallel machine and JSPs, this is much less the case on the FJSP. Some of the research on robustness measures for the FJSP was discussed in [Section 4.2](#).

Most other proposed approaches rely on simulation, in particular Monte Carlo simulation, to represent the uncertainty. [Mahdavi et al. \(2010\)](#) describe a simulation-based decision support system (DSS) for a stochastic FJSP, which relies on simulation results to identify potential incremental improvements of the criteria. Combining Monte Carlo simulation and a PSO algorithm, the approach in [Zhang et al. \(2012b\)](#) aims at minimizing the expected total weighted tardiness. As already discussed in [Section 6.1](#), [Doh et al. \(2013\)](#) consider the problem with uncertain job arrivals and processing times and machine breakdowns. [Mokhtari & Dadgar \(2015\)](#) consider the FJSP with machine maintenance operations, where processing times and due dates are stochastic and machine failure rates are time-varying. An MILP is introduced that minimizes the expected number of tardy jobs and satisfies a minimum total availability constraint. A simulation-optimization framework relying on SA and Monte Carlo simulation is also proposed. [Jamrus et al. \(2018\)](#) consider fuzzy processing times when minimizing the makespan by using PSO with a Cauchy distribution and genetic operators. More recently, [Flores Gómez et al. \(2021\)](#) use the notion of makespan service level to evaluate the quality of a solution in the FJSP with stochastic processing times. More precisely, the makespan service level is the probability that the schedule is completed before a given time. An approach combining TS and Monte Carlo simulation is presented to solve the problem.

6.4. Other characteristics

We have covered in the previous sections the major additional characteristics of FJSPs considered in the literature. Other characteristics less studied are discussed below, namely cyclic scheduling and variable processing times.

Cyclic scheduling In contrast to the problems previously discussed, cyclic scheduling assumes that each operation has to be processed infinitely. Applications can be found in compiler design, manufacturing and timetabling. Common criteria for acyclic scheduling are not relevant in cyclic scheduling, where the cycle time or the work in process are often minimized. [Quinton et al. \(2020\)](#) and [Bozejko et al. \(2017\)](#) minimize the cycle time for the cyclic FJSP using exact and heuristic approaches, respectively.

Variable processing times While a large majority of papers assume constant processing times, [Section 6.3](#) considers problems where some of the problem parameters, typically the processing times, are not deterministic. There is, however, significant research in scheduling that aims to model the variability of processing times, in particular learning and aging (or deterioration) effects. [Tayebi Araghi et al. \(2014\)](#) are among the few authors that consider learning and aging effects in the FJSP. These models are theoretically attractive but not often realistic, in particular regarding the learning effect that usually impacts processing times on a longer horizon than a scheduling horizon. Instead of being dependent variables, there are also contexts where processing times

are independent decision variables that should be optimized, in addition to the assignment and sequencing decisions. This is typically the case when machine speeds can be controlled to minimize the total energy cost or consumption, as discussed in [Section 4.2](#). Controllable processing times in the FJSP are typically modeled as bounded decision variables, where shorter processing times mean increased resource consumption. Such modeling is considered in [Lu et al. \(2017\)](#), where the makespan and the resource consumption, that is linear to the compression rate of the processing times, are minimized.

In the last three sections, different references are reviewed to discuss the modeling and the challenges of optimizing alternative and multiple criteria, integrating various constraints and considering additional characteristics in the FJSP. To further illustrate the diversity of the FJSP extensions, [Table 3](#) provides the three-field notation of some of the reviewed references. The references are chosen such that there is at least one reference for each of the notations introduced in [Table 1](#). Over half of the references solve practical problems encountered in batch or discrete part manufacturing, railway and healthcare.

7. Solution approaches

The complexity associated to the assignment decisions of machines to operations makes solution approaches significantly more complex to design for the FJSP than for the JSP. As for most combinatorial optimization problems, solution approaches for the FJSP and its various extensions can be classified into three main categories: Exact algorithms, heuristics and metaheuristics, that are respectively discussed in [Sections 7.1, 7.2 and 7.3](#).

7.1. Exact approaches

Branch and bound (B&B) algorithms are the main approaches considered to solve the FJSP to optimality, whether it is a dedicated algorithm or the problem modeled as a Mixed Integer Linear Program (MILP) and solved using a standard solver. As already mentioned in [Section 1](#), efficient B&B algorithms for the JSP rely on the resolution of single-machine problems, and cannot be extended to the FJSP as machines are flexible. MILPs are often proposed in papers dealing with the FJSP and its extensions. Although an MILP is useful to formalize the problem and its additional constraints, it can rarely lead to optimal solution or even good solutions with a standard solver when tackling instances of conventional sizes. This is in particular due to Constraints (4) and (5) in the mathematical model of [Section 2.1](#), which induce very poor linear relaxations. Generally speaking, this is the case when modeling assignment and sequencing decisions with integer linear constraints.

Applications of B&B can be found in the Flexible Manufacturing System (FMS) literature, as the FJSP resembles an FMS due to its routing and machine flexibility. [Berrada & Stecké \(1986\)](#) first discuss the loading problem in an FMS, and propose a B&B algorithm for assigning machine tools, operations and associated cutting tools to balance the workload on the machines. Further efficient B&B algorithms are developed for several FMS variants (see e.g., [Kim & Yano, 1994](#); [Lloyd et al., 1995](#)). To minimize makespan for an extension of FJSP encountered in rail car maintenance, [Hansmann et al. \(2014\)](#) propose a B&B algorithm that is competitive with commercial solvers. [Soto et al. \(2020\)](#) propose a parallel B&B to solve the multi-objective FJSP with the objective of minimizing the makespan, the maximum workload and the total workload. By taking advantage of the parallel implementation and the GA to initialize the upper bound, the approach found the optimal Pareto front for 12 instances from Fdata benchmark ([Fattahi et al., 2007](#)) and one instance from KCdata benchmark ([Kacem et al., 2002a](#)).

Table 3
 $\alpha|\beta|\gamma$ notation for some of the reviewed references.

Reference	α	β	γ
Dauzère-Pérès et al. (1998)	FSFMRJ	bom	$C_{\max}$
Brucker & Neyer (1998)	MMJ		$C_{\max}$
Raaymakers & Hoogeveen (2000)	FJ	nw, overlap	$C_{\max}$
Mason et al. (2002)	FJ	$p - \text{batch}$, incompatible, r_j , recrc, s	TWT
Kis (2003)	FSFPMMP	bom	$C_{\max}$
Alvarez-Valdes et al. (2005)	FJ	bom, h_{kl}^f , nw, overlap, ...	irreg
Gomes et al. (2005)	FJ	b , rcrc	JIT
Loukil et al. (2007)	FJ	bom, overlap	$C_{\max}$, $T_{\max}$, TWF TWT, WNTJ
Pham & Klinkert (2008)	MMJ	block, h_{kl}^f , s	$C_{\max}$
Vilcot & Billaut (2008)	FJ	bom, s	$C_{\max}$, $L_{\max}$
De Giovanni & Pezzella (2010)	DFJ		$C_{\max}$
Grobler et al. (2010)	FMRJ	h_{kl}^f , res, s	$C_{\max}$, JIT
Gröflin et al. (2011)	FJ	block, $d_{it}^{kk'}$, s	$C_{\max}$
Mati & Xie (2011)	MMJ	block	$C_{\max}$
Mati et al. (2011)	FJ	block, rcrc	$C_{\max}$
Liu & Kozan (2011)	FJ	block, nwt	$C_{\max}$
Özgüven et al. (2012)	FPFJ	s	$C_{\max}$, W_M , W_T
Yugma et al. (2012)	FJ	$d_{it}^{kk'}$, h_{kl}^f , $p - \text{batch}$, incompatible, r_j , s	reg, irreg
Gomes et al. (2013)	FJ	bom, rcrc	JIT
Xiong et al. (2013)	FJ	h_{kl}^f , res, r_j	$C_{\max}$, QR
Zambrano Rey et al. (2014)	FJ	$d_{it}^{kk'} \geq 0$	E/T
Shen & Yao (2015)	FJ	h_{kl}^f , res, r_j	$C_{\max}$, SR, TWT, W_M
Thörnblad et al. (2015)	FJ	$d_{it}^{kk'} \geq 0$, h_{kl}^w	TWF, TWT
Ham & Cakici (2016)	FJ	$p - \text{batch}$, compatible	$C_{\max}$
Knopp et al. (2017)	FJ	$p - \text{batch}$, incompatible, r_j , recrc, s	reg
Burdett & Kozan (2018)	FJ	block, $d_{it}^{kk'}$, h_{kl}^f , nwt, r_j , s , recrc, ...	$C_{\max}$
Piroozfard et al. (2018)	FJ		CF, TWT
Shen et al. (2018)	FJ	s	$C_{\max}$
Lunardi et al. (2021)	FJ	bom, h_{kl}^f , overlap, res, s	$C_{\max}$
Park & Ham (2022)	FJ	h_{kl}^f	$C_{\max}$, EC
Tamssaouet et al. (2022)	FJ	$d_{it}^{kk'} \geq 0$, h_{kl}^f , $p - \text{batch}$, incompatible, r_j , rcrc, s	reg, irreg

Most other studies on the FJSP focus on developing MILPs. For the classical FJSP, Özgüven et al. (2010) propose MILPs and compare their performances, while Roshanaei et al. (2013) present two MILPs based on positions and sequences, respectively. MILPs are also the choice approaches used to solve extensions of the FJSP not yet studied in the literature. Regarding extension in the α field, Meng et al. (2020) and Özgüven et al. (2010) propose MILP for the distributed FJSP ($\alpha = DFJ$) and for the FJSP with process plan flexibility ($\alpha = FPFJ$). Examples of extensions in the β field are: Limited buffer ($\beta = b$) in Gomes et al. (2005), non-linear routes ($\beta = bom$) in Birgin et al. (2014), flexible unavailability periods ($\beta = h_{kl}^w$) in Thörnblad et al. (2015), parallel batching ($\beta = p - \text{batch}$) in Ham & Cakici (2016). While most of the MILPs consider regular criteria, some consider non-regular ones such as energy cost ($\gamma = EC$) in Park & Ham (2022) and $\gamma = JIT$ in Gomes et al. (2013). Moreover, MILPs are also used to solve FJSP integrated with other problems: Vehicle routing problem in Mohammadi et al. (2020) and supply chain planning in Ceylan et al. (2021).

Constraint Programming (CP) is another exact approach that has been getting more attention in recent years to solve the FJSP and its extensions. All the numerical results published in recent works show the superiority of CP to MILP models by optimally solving a fraction of medium-size instances and delivering feasible solutions for large instances appearing in practice. CP models are often proposed for extensions of the FJSP: Distributed FJSP ($\alpha = DFJ$) in Meng et al. (2020), parallel batching ($\beta = p - \text{batch}$) in Ham & Cakici (2016), non-linear routes ($\beta = bom$) in Lunardi et al. (2020) and Kasapidis et al. (2021), time lags ($\beta = d_{it}^{kk'}$) in Boyer et al. (2021) and multiple resources in Kasapidis et al. (2023). Despite the significant advances in the performance of CP solvers, most studies also assert the superiority of metaheuristics, to be discussed in Section 7.3, in finding better quality solutions in reasonable computational times.

7.2. Heuristics

Before covering metaheuristics in Section 7.3, simpler and fast heuristics are discussed in this section, first relying on dispatching rules and then specific to the problem. Note that many papers proposing metaheuristics also include fast heuristics to determine initial schedules.

Earlier work intensively utilizes dispatching rules and analyzes their performance (Jeong & Kim, 1998; O'Keefe & Kasirajan, 1992; Shanker & Tzen, 1985). The application of dispatching rules can also be found in recent papers, especially for the multi-objective FJSP.

Shi-Jin et al. (2008) propose a filtered-beam-search-based (FBSB) heuristic for the FJSP to minimize the makespan, the total workload of machines and the workload of the most loaded machines. The FBSB integrates dispatching rule-based heuristics. Later, Wang & Yu (2010) further develop the FBSB heuristic to deal with the FJSP with maintenance activities. Composite dispatching rules based on genetic programming are proposed in Tay & Ho (2008) to solve the multi-objective FJSP, while Calleja & Pastor (2014) present an algorithm with priority dispatching rules to solve a real-world case of the FJSP with transfer batches. It is also worth mentioning that in Ortíz et al. (2018), the dispatching rule outperforms swarm intelligence algorithms in a practical textile manufacturing system.

Besides dispatching rules, traditional heuristics are extensively applied to the FJSP (Alvarez-Valdes et al., 2005; Chang et al., 1989; Mati et al., 2001). These heuristics are usually more refined than dispatching rules and show better solution quality. In order to further improve the performance of heuristics, researchers have investigated and integrated more specified and advanced components. These approaches often adopt key concepts of some metaheuristics while keeping a clear and manageable structure.

Fattahi et al. (2007) develop both integrated and hierarchical approaches to solve the FJSP, where six hybrid search structures that depend on the selected search approach and heuristics are proposed. Mejia & Odrey (2005) present a Petri net-based algorithm, known as A* algorithm, which selectively expands portions of a Petri net reachability graph to find a better schedule. Combining with this algorithm, Huang et al. (2008) develop a hybrid heuristic search strategy. The search scheme is able to control the search result and reach an early termination condition. Moreover, new heuristic functions for the A* algorithm are proposed in Lee & Lee (2010) to improve efficiency. Based on harmony search and large neighbourhood search, Yuan & Xu (2013b) design a hybrid two-stage search heuristic for the large-scale FJSP with makespan criterion.

Again because machines are flexible, the definition of a bottleneck machine in the shifting bottleneck heuristic for the JSP is also not as clear in the FJSP. However, by dividing the disjunctive graph representation, Liu & Kozan (2012) propose a hybrid shifting bottleneck procedure for the FJSP with parallel machines. The procedure includes a modified Carlier's algorithm and Jackson's rule, as well as TS functions. Also for this setting extended by re-entrancy, efficient algorithms for the special case of two centers/machines with an absolute worst-case bound are developed in Kubiak et al. (2020).

For the multi-objective FJSP, Gao et al. (2015) present four heuristics to schedule the FJSP with new job insertion. The objectives are to minimize the makespan, an average of earliness and tardiness, maximum machine workload and total machine workload. Pérez & Raupp (2016) propose a new hierarchical heuristic, which is an adaptation of Newton's method for solving continuous multi-objective unconstrained optimization problems. Ozturk et al. (2019) introduce a hybrid priority scheduling rule combined with the cellular system for the multi-objective and dynamic FJSP.

In addition, for a broad variety of settings of the FJSP, specific heuristic approaches can be found in Baruwala et al. (2014); Bekkar et al. (2019) and Sobeyko & Mönch (2016).

7.3. Metaheuristics

When machine flexibility is very high, i.e., when operations can be performed on many machines, finding very good or even optimal solutions is easier. The benchmark instances of Hurink et al. (1994) discussed in Section 3 are a well-known example, where the most flexible instances *HUdata/vdata* are easier to solve compared to instances *HUdata/edata* and *HUdata/rdata*. This can be explained by the fact that the total number of solutions increases with machine flexibility, but also the number of very good solutions which are thus easier to reach.

To solve the most challenging FJSP instances, advanced metaheuristics are the most appropriate approaches. As mentioned above, the number of solutions to explore is huge, and major contributions in metaheuristics are focusing on:

- Deriving important structural properties to guide the search process in relevant and promising solution areas;
- Integrating metaheuristic components that are extended and improved specifically for the FJSP.

Regarding the first point, to save the computational cost of exploring new solutions from a given solution in a neighborhood structure, conditions have been proposed to ensure the feasibility and quality of solutions without explicitly exploring them. Section 7.3.1 first presents well-known feasibility conditions of a move in the integrated neighborhood structure of Dauzère-Pérès & Paulli (1997) that are valid for any criterion. Functions to evaluate the quality of moves for the makespan criterion can be found in Section 2 of the supplementary materials. Afterwards,

trajectory-based metaheuristics are discussed. Section 7.3.2 covers population-based metaheuristics and Section 7.3.3 metaheuristics combining both types of metaheuristics.

7.3.1. Trajectory-based metaheuristics

Algorithms working on single solutions, mainly local search approaches, form a search trajectory. Trajectory-based metaheuristics are widely applied to solve FJSPs, including Simulated Annealing (SA), Tabu Search (TS) and Variable Neighbourhood Search (VNS). The choice of the neighborhood structure in local search approaches is critical. Initially, the moves to resequence an operation, which are similar in the JSP, were separated from the moves to reassign an operation, typically in an iterative process. Integrated moves not differentiating between resequencing and reassigning an operation were shown to be much more effective.

It is important to point out that the general feasibility properties are embedded in some of the most efficient trajectory-based or hybrid metaheuristics. These properties ensure the feasibility of solutions that can be reached in a neighborhood structure without actually performing the move.

The integrated reassignment and resequencing move is first introduced in Dauzère-Pérès & Paulli (1997) with the following theorem.

Theorem 1. Moving operation i between operations u with $u \neq fr(i)$ and v with $v \neq pr(i)$, $(u, v) \in \mathcal{E}$ does not lead to infeasible solutions if:

1. $h_u < h_{fr(i)} + p_{fr(i)}^{a(fr(i))}$, and
2. $h_v + p_v^{a(v)} > h_{pr(i)}$.

Compared to Theorem 1, Theorem 2 in Shen et al. (2018) below is less restrictive and allows more feasible moves.

Theorem 2. Moving operation i between operations u with $u \neq fr(i)$ and v with $v \neq pr(i)$, $(u, v) \in \mathcal{E}$ does not lead to infeasible solutions if:

1. $\max(h_{ps(u)}, h_{fr(u)}) < h_{fr(i)} + p_{fr(i)}^{a(fr(i))}$, and
2. $\min(h_{fs(v)} + p_{fs(v)}^{a(fs(v))}, h_{fr(v)} + p_{fr(v)}^{a(fr(v))}) > h_{pr(i)}$.

For the FJSP with job routing flexibility, the conditions are extended in Dauzère-Pérès et al. (1998) as follows:

1. $h_u < \min_{o \in \mathcal{F}(i) \setminus fs_k(i)} h_o + p_o^{a(o)}$, and
2. $h_v + p_v^{a(v)} > \max_{o \in \mathcal{P}(i) \setminus ps_k(i)} h_o$.

Mastrolilli & Gambardella (2000) restricts the integrated move of Dauzère-Pérès & Paulli (1997) to k -insertions, where k is the machine on which operation i is resequenced or reassigned. A k -insertion relies on two sets $\mathcal{R}_k^+$ and $\mathcal{R}_k^-$ for the current operation sequence π_k :

$$\mathcal{R}_k^+ = \{i \in \pi_k | h_i + p_i^{a(i)} > h_{pr(i)} + p_{pr(i)}^{a(pr(i))}\}, \quad (17)$$

$$\mathcal{R}_k^- = \{i \in \pi_k | p_i^{a(i)} + q_i > p_{fr(i)}^{a(fr(i))} + q_{fr(i)}\}. \quad (18)$$

The following theorem in Mastrolilli & Gambardella (2000) ensures the feasibility of k -insertions.

Theorem 3. A set of feasible solutions is obtained by inserting operation i after every operation of $\mathcal{R}_k^- \setminus \mathcal{R}_k^+$ and before every operation of $\mathcal{R}_k^+ \setminus \mathcal{R}_k^-$.

Tamssaouet & Dauzère-Pérès (2023) propose a neighborhood structure framework that generalizes, among others, the results of Dauzère-Pérès & Paulli (1997); Mastrolilli & Gambardella (2000)

and Shen et al. (2018) related to the feasibility and quality evaluations for any regular criterion. The proposed procedures are parameterized, making it possible to control the trade-off between the efficiency and the accuracy of the evaluations.

In Kis (2003), an extended version of the FJSP with complex routes is considered (see Section 6.1) as well as with multiple resources (see Section 6.2). Instead of moving a single operation as in the above references, the approach of Kis (2003) allows a set of operations to be inserted in a partial schedule. An insertion of operation i in the current sequence π_k between u and v can be specified as subsets $\mathcal{H}$ with all operations preceding i and subsets $\bar{\mathcal{H}}$ with all operations succeeding i . The following theorem gives necessary conditions for a feasible insertion in this extended case.

Theorem 4. A subset $\mathcal{H}$ is feasible if and only if

1. $\mathcal{P}(i) \subseteq \mathcal{H}$,
2. $\mathcal{H} \cap \mathcal{F}(i) = \emptyset$, and
3. $u \in \mathcal{H}$ implies $v \in \mathcal{H}$ for all $(v, u) \in \mathcal{E} \cup \mathcal{A}$.

We next address the individual trajectory-based methods applied in this field. Early research relied on TS with fruitful results (Brandimarte, 1993; Dauzère-Pérès & Paulli, 1997; Hurink et al., 1994). TS is still successfully used in recent studies (Hajibabaei & Behnamian, 2021; Shen et al., 2018).

Regarding the application of SA to the FJSP, various extensions of SA are proposed in addition to the standard framework, such as linguistic and priority rules in Baykasoglu (2002), stochastic local search with probabilistic moves in Yazdani et al. (2009) and an acceleration mechanism with partial scheduling in Cruz-Chávez et al. (2017).

As trajectory methods heavily rely on neighbourhood structures, VNS also plays a prominent role in the research on the FJSP. VNS algorithms include a parallel VNS in Yazdani et al. (2010), a sequential VNS separated for subproblems in Lei & Guo (2014), as well as a combined version of VNS in Karimi et al. (2012). Other applications of trajectory methods are GRASP (Boyer et al., 2021; Knopp et al., 2017), scatter search (González et al., 2015), etc.

Considering again neighborhood structures, Bozejko et al. (2010) mainly focus on the reassignment of operations and define a transfer move type (t -move). González et al. (2015) propose neighbourhood structures N^π and N^π , where operations are moved depending on their positions in the corresponding block. Furthermore, neighborhood structure N^α is developed to reassign operations by taking a new compatible machine into account. In addition, extended neighborhood structures N_2^π and N_2^α are defined for path relinking. Similar neighbourhood functions can be found in Kemmoé-Tchomté et al. (2017). The authors further classify the moves performed on general operations, critical operations, as well as the frequency of moves.

The intensified and diverse neighbourhood functions, on the other hand, raise the concern whether the proposed neighbourhoods are optimum connected. A few studies have explicitly discussed the connectivity properties (e.g., Dauzère-Pérès & Paulli, 1997; Mastrolilli & Gambardella, 2000). As mentioned earlier, theories integrated in the neighbourhoods, such as feasibility properties as well as different methods for evaluating potential moves, contribute significantly to solution efficiency. However, sophisticated mechanism may also impair neighbourhood connectivity. To further improve the competitiveness of metaheuristics, it becomes increasingly important to address connectivity issues.

7.3.2. Population-based metaheuristics

Different from trajectory-based metaheuristics, population-based metaheuristics deal with a set of individual solutions. We discuss the main components of a Genetic Algorithm, which is the most popular population-based metaheuristic for the FJSP.

- **Encoding and decoding.** A two-vector representation, a machine assignment vector and an operation sequence vector, is commonly used for the FJSP. Initially proposed for the JSP, Gao et al. (2008) introduce a priority-based decoding algorithm to translate chromosomes into active schedules. Each operation is scheduled on its assigned machine according to the order of the operation sequence vector. This representation guarantees solution feasibility. An alternative encoding scheme for the JSP is the permutation representation (Bierwirth et al., 1996). Similarly, Driss et al. (2015) propose a chromosome representation known as permutation job, which is a coding scheme for individual jobs that considers all constraints of the FJSP.
- **Selection operator.** In general, chromosomes satisfying the fitness criteria are likely to be selected for the next generation. Tournament selection is often used for the FJSP, e.g., Lei (2010), but also roulette wheel selection e.g., Chan et al. (2006a); Pezzella et al. (2008) adopt three selection strategies: Binary tournament, n -size tournament, and linear ranking. De Giovanni & Pezzella (2010) use the linear ranking selection strategy, while Wan et al. (2010) integrate the binary tournament selection and the linear ranking selection strategies. As another alternative, an elitism selection strategy and its variants can be found in Moon et al. (2008).
- **Crossover operator.** As the simplest crossover operators, one-point crossover (OPX), e.g., Moon et al. (2008), as well as two-point crossover (TPX), e.g., De Giovanni & Pezzella (2010), are popular for solving FJSPs. Extensions of TPX such as Precedence Preserving order-based crossover (PPX), e.g., Lei (2010), or Partially Mapped crossover (PMX), e.g., Mati et al. (2011), are also used. Finally, assignment crossover (ACX), that only deals with the assignment properties of the individuals, is considered, e.g., Defersha & Chen (2010).
- **Mutation operator.** Swap mutation, e.g., Zhang & Yang (2016), and random mutation, e.g., Mati et al. (2011), are classical operators. Specific to the FJSP, the assignment mutation operator is for instance considered in Zhang et al. (2012a).

Besides GAs, Particle Swarm Optimization (PSO), which also maintains a population of candidate solutions, is also popular for solving FJSPs. One representative variant of discrete PSO is developed in Ding & Gu (2020) with a chain-based encoding scheme, that converts the FJSP to an appropriate operation linked list, and a decoding scheme that explores the solution space. To update the particles' position, Gu et al. (2020) adopt different operators to imitate the optimization mechanism of the basic PSO algorithm.

Rossi & Dini (2007) are the first to use Ant colony optimization (ACO) to tackle the FJSP, where routing flexibility and separable setup times are considered. Later, Huang et al. (2013) develop a two-phormone ACO algorithm for the FJSP with due window and SDSTs, while Rossi (2014) presents an ACO algorithm with reinforced phormone relationships for the FJSP with SDSTs and transportation times, while a knowledge-based ACO algorithm is proposed in Xing et al. (2010). Different from the above GAs and PSOs, the general idea to apply ACO is to construct a conjunctive graph as discussed in Section 2.2.

Estimation of Distribution Algorithms (EDA) are comparatively new in the field of evolutionary computation. As an example, Wang et al. (2012a) propose a bi-population based EDA with a probability model, different search operators and an updating mechanism for generating new solutions.

7.3.3. Hybrid metaheuristics

To combine the ability of trajectory-based metaheuristics for search intensification and of population-based metaheuristics for search diversification, the hybridization of both types of metaheuristics has shown to be successful for many combinatorial

optimization problems. Some population-based metaheuristics discussed in Section 7.3.2 are in fact hybrid metaheuristics as they incorporate local search components.

We next classify major categories of hybrid metaheuristics for the FJSP. Generally speaking, whenever hybridization is performed, the key is to take full advantage of the different types of metaheuristics by integrating a variety of components into a basic metaheuristic framework. However, one important point rarely discussed in papers introducing hybrid metaheuristics is the impact of each component on the solution quality. We believe that it is critical to propose efficient algorithms of reasonable complexity.

- **GA with trajectory-based methods.** A variety of hybrid metaheuristics based on GA as the outlier framework are available in the literature, e.g., Gao et al. (2008). In particular, Zribi et al. (2007) propose a hierarchical hybrid GA to deal with machine assignment and operation sequencing. More recently, a variable neighbourhood descent GA is developed in Liu et al. (2021), where a PSO-based mutation operator and a VND-based neighbourhood structure are embedded in the standard GA framework.
- **Further biology-inspired metaheuristics with trajectory-based methods.** PSO is often hybridized with TS, SA or other local search heuristics for multi-objective FJSPs, e.g., Zhang et al. (2009). Another example can be found in Jamrus et al. (2018), where a hybrid approach is developed with a PSO, a Cauchy distribution, and genetic operators to find a job sequence with uncertain processing times. Kato et al. (2018) present another hybrid metaheuristic with a hierarchical approach combining PSO and a random restart hill climbing. The hybridization of Artificial Bee Colony (ABC) and a local search mechanism is proposed for instance in Wang et al. (2013). ACO is hybridized with a multi-agent system negotiation in Zhang & Wong (2017), whereas El Khoukhi et al. (2017) propose an ACO with a dynamic history and dual-task ants, where a local search and a set of dispatching rules are integrated.
- **Other hybrid metaheuristics** include scatter search with path re-linking and TS in González et al. (2015), harmony search with local search in Yuan et al. (2013), as well as a Jaya algorithm hybridized with TS in Fan et al. (2021).

Overall, the applications of (hybrid) metaheuristics to solve FJSPs are vast. Additional studies can be found in Section 3 in the supplementary materials.

8. Conclusions and future research directions

This review paper covers the flexible job shop scheduling problem (FJSP). The different criteria, constraints, configurations and solution approaches studied in the literature are discussed, with an emphasis on the major results and findings. Several interesting conclusions can be drawn:

- Although the makespan is the most studied criterion in the literature, it seems that it is rarely relevant in practical settings. On the other hand, considering multiple criteria is often necessary.
- As neighborhoods to explore are often very large, because of the flexibility of machines, efficient local search approaches rely on move feasibility and evaluation procedures to avoid the computational cost of checking all possible moves.
- Some constraints are more difficult to handle than others, in particular constraints that might lead to infeasible solutions, such as maximum time lags and blocking constraints.

We offer below various perspectives related to different aspects of the FJSP.

- **Non-regular criteria.** Various non-regular criteria are relevant to study as discussed in Section 4.2. An interesting perspective is to consider the realistic case where an operation cannot wait if its assigned machine is available. In this case, a unique schedule is associated with a given assignment and sequence of operations, which makes the problem easier to solve. However, semi-active schedules might not be optimal. More generally, heuristic or exact algorithms should be designed to optimize the timing decisions. Efficient neighborhood structures and insertion methods are highly relevant to develop efficient heuristics that take the properties of non-regular criteria into account. Results from other scheduling problems could be exploited. For example, the approach for the JSP introduced in Bürgy & Bülbül (2018) minimizes the sum of separable convex cost functions attached to the operation start times and the differences between the start times of arbitrary pairs of operations. Various criteria (earliness and tardiness costs, intermediate inventory holding costs) can be written in this form. There is also a rich literature dealing with non-regular criteria in the Resource Constrained Project Scheduling Problem (RCPSp), see e.g., Neumann et al. (2003).
- **Stochastic and dynamic FJSP.** Significant research is still needed to handle random and dynamic events in scheduling problems. This is particularly relevant in the FJSP, as for instance (1) The processing time of an operation might be more uncertain on a machine than on another, and (2) Reassigning an operation to another machine is possible if a machine breaks down. Moreover, many practical settings are dynamic, in the sense that an optimized schedule is computed on a rolling horizon, i.e., regularly recomputed to integrate the arrival of new jobs or the changing status of machines.
- **Machine degradation.** More and more critical in modern manufacturing systems is the consideration of the aging effect on machine efficiency. The processing times of an operation become variables that depend on the scheduling decisions. Moreover, some operations might require their assigned machines not to be too “old”. Hence, in the FJSP, the assignment of machines becomes more constrained over time. The planning of maintenance operations might also be scheduled to restore the “health” of machines.
- **Multiple resources.** Although it is the case in various application contexts, the consideration of multiple flexible resources has not been studied much in the FJSP. The first solution approach proposed in Dauzère-Pérès et al. (1998) should be challenged, by considering the reassignment of an operation on all its necessary resources in one move, as in Kis (2003), or some additional constraints such as incompatible resources or non-synchronized resources, as in Dauzère-Pérès & Pavageau (2003). Another interesting and relevant constraint is when one of the resources should be assigned for multiple consecutive operations in the route, as in Knopp et al. (2014). This is typically the case for a transportation resource, or more generally for a critical resource that holds the job, which must remain the same for several consecutive operations (e.g., fixtures in Thörnblad et al., 2015, and hospital beds in hospitals Burdett & Kozan, 2018).
- **Applicability of solution approaches.** Given the complexity of the FJSP and its extensions, it is unlikely that simple approaches can perform practically well. However, it is essential that the proposed approaches are as streamlined as possible and only incorporate components that make a difference. Also, the efficiency of the search process should always be an important criterion in evaluating a solution approach. As such approaches are part of systems designed to support decision making at the operational level, which are thus used at a high frequency, short computational times are required. The dynamic aspect of

most industrial environments also requires the integration of rescheduling considerations into solution approaches. The optimization of schedule robustness or stability measures and the design of effective and efficient job insertion algorithms are examples of relevant issues to study.

- **Analysis of solution approaches and relevant benchmark instances.** From a more conceptual point of view, understanding the important characteristics that make a solution approach effective would be interesting to study. The analysis could be theoretical, for instance, on the properties of various neighborhood structures or moves, but also empirical by constructing sets of test instances that extend the current well-known benchmarks. The goal would be to better define instances that differ for instance by the machine flexibility, the range of the processing times on the machines for each operation or the relations between the machines. Regarding the last point, as in the parallel machine scheduling problems, machines could be identical, related or unrelated, although extending these definitions is not straightforward in the FJSP.
- **Calling upon generic approaches.** The majority of work dealing with the FJSP (and of scheduling work in general) focuses on developing approaches that are tailored to a specific problem, for example, FJSP with makespan. It is often assumed that the insights gained from the understanding of a specific problem can help extending those approaches to other problems. Except for a few works that attempt to provide approaches for quite important and generic extensions of the FJSP (Dauzère-Pérès et al., 1998; García-León et al., 2019; Kis, 2003), relying only on this “bottom-up” approach will probably not support the emergence of one or few generic approaches. We believe that having a stream of research that explores a more “top-down” strategy will benefit the field of scheduling, by providing ready-to-use approaches for some generic extensions of the FJSP. For example, designing an approach that can solve efficiently and effectively $FSMMJ|block, bom, d_{ij}^{kk'}|reg$ would be a significant achievement. Such a problem generalizes most of the machine scheduling problems studied in the literature. By already tackling the most difficult features, tailoring such an approach for a specific context would be easier than what is currently investigated. Instead of simple dispatching rules or other low-performance approaches, a solution approach for such a generic problem can be used as a benchmark to assess new and tailored approaches for simpler problems.

To conclude, by allowing operations to be assigned to multiple machines, the flexible job shop scheduling problem has opened very rich theoretical and applied research, with still numerous research avenues to explore. An important question is which of these avenues are the most relevant. We hope that this review provides some of them and the keys to investigate the FJSP.

Acknowledgments

The authors would like to thank their many colleagues who have contributed to this review through discussion and exchange of ideas.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.ejor.2023.05.017.

References

Abdelmaguid, T. F. (2015). A neighborhood search function for flexible job shop scheduling with separable sequence-dependent setup times. *Applied Mathematics and Computation*, 260, 188–203.

- Adams, J., Balas, E., & Zawack, D. (1988). The shifting bottleneck procedure for job shop scheduling. *Management Science*, 34, 391–401.
- Ahmadi, E., Zandieh, M., Farrokh, M., & Emami, S. M. (2016). A multi objective optimization approach for flexible job shop scheduling problem under random machine breakdown by evolutionary algorithms. *Computers and Operations Research*, 73, 56–66.
- Al-Hinai, N., & Elmekawy, T. Y. (2011). Robust and stable flexible job shop scheduling with random machine breakdowns using a hybrid genetic algorithm. *International Journal of Production Economics*, 132(2), 279–291.
- Allahverdi, A. (2015). The third comprehensive survey on scheduling problems with setup times/costs. *European Journal of Operational Research*, 246, 345–378.
- Alvarez-Valdes, R., Fuentes, A., Tamarit, J. M., Giménez, G., & Ramos, R. (2005). A heuristic to schedule flexible job-shop in a glass factory. *European Journal of Operational Research*, 165(2), 525–534.
- Andrade-Pineda, J. L., Canca, D., Gonzalez-R, P. L., & Calle, M. (2020). Scheduling a dual-resource flexible job shop with makespan and due date-related criteria. *Annals of Operations Research*, 291(1–2), 5–35.
- Aytug, H., Lawley, M. A., McKay, K., Mohan, S., & Uzsoy, R. (2005). Executing production schedules in the face of uncertainties: A review and some future directions. *European Journal of Operational Research*, 161(1), 86–110.
- Barnes, J. W., & Chambers, J. B. (1996). Flexible job shop scheduling by tabu search. *Technical Report*. University of Texas, Austin.
- Baruwa, O. T., Piera, M. A., & Guasch, A. (2014). Deadlock-free scheduling method for flexible manufacturing systems based on timed colored petri nets and any-time heuristic search. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 45(5), 831–846.
- Baykasoglu, A. (2002). Linguistic-based meta-heuristic optimization model for flexible job shop scheduling. *International Journal of Production Research*, 40(17), 4523–4543.
- Bekkar, A., Belalem, G., & Beldjilali, B. (2019). Iterated greedy insertion approaches for the flexible job shop scheduling problem with transportation times constraint. *International Journal of Manufacturing Research*, 14(1), 43–66.
- Benjaafar, S., & Ramakrishnan, R. (1996). Modelling, measurement and evaluation of sequencing flexibility in manufacturing systems. *International journal of production research*, 34(5), 1195–1220.
- Berrada, M., & Stecke, K. E. (1986). A branch and bound approach for machine load balancing in flexible manufacturing systems. *Management Science*, 32(10), 1316–1335.
- Bezoui, M., Olteanu, A.-I., & Sevaux, M. (2023). Integrating preferences within multi-objective flexible job shop scheduling. *European Journal of Operational Research*, 305(3), 1079–1086.
- Bierwirth, C., Mattfeld, D. C., & Kopfer, H. (1996). On permutation representations for scheduling problems. In *International conference on parallel problem solving from nature* (pp. 310–318). Springer.
- Birgin, E. G., Feofiloff, P., Fernandes, C. G., De Melo, E. L., Oshiro, M. T. I., & Ronconi, D. P. (2014). A MILP model for an extended version of the flexible job shop problem. *Optimization Letters*, 8(4), 1417–1431.
- Birgin, E. G., Ferreira, J. E., & Ronconi, D. P. (2015). List scheduling and beam search methods for the flexible job shop scheduling problem with sequencing flexibility. *European Journal of Operational Research*, 247(2), 421–440.
- Bissoli, D. C., Zufferey, N., & Amaral, A. R. S. (2021). Lexicographic optimization-based clustering search metaheuristic for the multiobjective flexible job shop scheduling problem. *International Transactions in Operational Research*, 28(5), 2733–2758.
- Boyer, V., Vallikavungal, J., Cantú Rodríguez, X., & Salazar-Aguilar, M. A. (2021). The generalized flexible job shop scheduling problem. *Computers and Industrial Engineering*, 160(June), Article 107542.
- Bozejko, W., Pempera, J., & Wodecki, M. (2017). A fine-grained parallel algorithm for the cyclic flexible job shop problem. *Archives of Control Sciences*, 27(2), 169–181. <https://doi.org/10.1515/acsc-2017-0010>.
- Bozejko, W., Uchroński, M., & Wodecki, M. (2010). Parallel hybrid metaheuristics for the flexible job shop problem. *Computers and Industrial Engineering*, 59(2), 323–333.
- Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 41(3), 157–183.
- Brucker, P., & Knust, S. (2012). *Complex scheduling*. Springer.
- Brucker, P., & Neyer, J. (1998). Tabu-search for the multi-mode job-shop problem. *Operations-Research-Spektrum*, 20(1), 21–28.
- Brucker, P., & Schlie, R. (1990). Job-shop scheduling with multi-purpose machines. *Computing*, 45(4), 369–375.
- Burdett, R. L., Corry, P., Eustace, C., & Smith, S. (2020). A flexible job shop scheduling approach with operators for coal export terminals - A mature approach. *Computers and Operations Research*, 115, 104834.
- Burdett, R. L., & Kozan, E. (2010). A disjunctive graph model and framework for constructing new train schedules. *European Journal of Operational Research*, 200(1), 85–98.
- Burdett, R. L., & Kozan, E. (2018). An integrated approach for scheduling health care activities in a hospital. *European Journal of Operational Research*, 264(2), 756–773.
- Bürge, R., & Bülbül, K. (2018). The job shop scheduling problem with convex costs. *European Journal of Operational Research*, 268, 82–100.
- Caldeira, R., & Gnanavelbabu, A. (2021). Solving the flexible job shop scheduling problem using an effective Jaya algorithm. In *Advances in materials and manufacturing engineering* (pp. 125–132). Springer.

- Caldeira, R. H., & Gnanavelbabu, A. (2019). Solving the flexible job shop scheduling problem using an improved Jaya algorithm. *Computers and Industrial Engineering*, 137, 106064.
- Calleja, G., & Pastor, R. (2014). A dispatching algorithm for flexible job-shop scheduling with transfer batches: An industrial application. *Production Planning and Control*, 25(2), 93–109.
- Cao, Z. C., Lin, C. R., & Zhou, M. C. (2021). A knowledge-based cuckoo search algorithm to schedule a flexible job shop with sequencing flexibility. *IEEE Transactions on Automation Science and Engineering*, 18(1), 56–69.
- Carlier, J., & Pinson, E. (1989). An algorithm for solving the job-shop problem. *Management Science*, 35(2), 164–176.
- Ceylan, Z., Tozan, H., & Bulkan, S. (2021). A coordinated scheduling problem for the supply chain in a flexible job shop machine environment. *Operational Research*, 21(2), 875–900.
- Chan, F., Wong, T. C., & Chan, L. Y. (2006a). Flexible job-shop scheduling problem under resource constraints. *International Journal of Production Research*, 44(11), 2071–2089.
- Chan, F. T. S., Chung, S. H., & Chan, P. L. Y. (2006b). Application of genetic algorithms with dominant genes in a distributed scheduling problem in flexible manufacturing systems. *International Journal of Production Research*, 44(3), 523–543.
- Chang, Y.-L., Matsuo, H., & Sullivan, R. S. (1989). A bottleneck-based beam search for job scheduling in a flexible manufacturing system. *International Journal of Production Research*, 27(11), 1949–1961.
- Chaudhry, I. A., & Khan, A. A. (2016). A research survey: Review of flexible job shop scheduling techniques. *International Transactions in Operational Research*, 23(3), 551–591.
- Chen, R., Yang, B., Li, S., & Wang, S. (2020). A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem. *Computers and Industrial Engineering*, 149, 106778.
- Cruz-Chávez, M. A., Martínez-Rangel, M. G., & Cruz-Rosales, M. H. (2017). Accelerated simulated annealing algorithm applied to the flexible job shop scheduling problem. *International Transactions in Operational Research*, 24(5), 1119–1137.
- Dauzère-Pérès, S., & Paulli, J. (1997). An integrated approach for modeling and solving the general multiprocessor job-shop scheduling problem using tabu search. *Annals of Operations Research*, 70(0), 281–306.
- Dauzère-Pérès, S., & Pavageau, C. (2003). Extensions of an integrated approach for multi-resource shop scheduling. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 33(2), 207–213.
- Dauzère-Pérès, S., Roux, W., & Lasserre, J.-B. (1998). Multi-resource shop scheduling with resource-flexibility. *European Journal of Operational Research*, 107, 289–305.
- De Giovanni, L., & Pezzella, F. (2010). An improved genetic algorithm for the distributed and flexible job-shop scheduling problem. *European Journal of Operational Research*, 200(2), 395–408.
- Defersha, F. M., & Chen, M. (2010). A parallel genetic algorithm for a flexible job-shop scheduling problem with sequence dependent setups. *The International Journal of Advanced Manufacturing Technology*, 49(1–4), 263–279.
- Defersha, F. M., & Rooyani, D. (2020). An efficient two-stage genetic algorithm for a flexible job-shop scheduling problem with sequence dependent attached/detached setup, machine release date and lag-time. *Computers and Industrial Engineering*, 147(June), 106605.
- Deliktas, D., Torkul, O., & Ustun, O. (2019). A flexible job shop cell scheduling with sequence-dependent family setup times and intercellular transportation times using conic scalarization method. *International Transactions in Operational Research*, 26(6), 2410–2431.
- Demirkol, E., Mehta, S., & Uzsoy, R. (1998). Benchmarks for shop scheduling problems. *European Journal of Operational Research*, 109, 137–141.
- Denkena, B., Schinkel, F., Pirnay, J., & Wilmseier, S. (2021). Quantum algorithms for process parallel flexible job shop scheduling. *CIRP Journal of Manufacturing Science and Technology*, 33, 100–114.
- Deroussi, L. (2014). A hybrid PSO applied to the flexible job shop with transport. In *International conference on swarm intelligence based optimization* (pp. 115–122). Springer.
- Deroussi, L., & Norre, S. (2010). Simultaneous scheduling of machines and vehicles for the flexible job shop problem. In *International conference on metaheuristics and nature inspired computing, Djerba Island Tunisia* (pp. 1–2).
- Ding, H., & Gu, X. (2020). Improved particle swarm optimization algorithm based novel encoding and decoding schemes for flexible job shop scheduling problem. *Computers and Operations Research*, 121, 104951.
- Ding, J., Lü, Z., Li, C.-M., Shen, L., Xu, L., & Glover, F. (2019). A two-individual based evolutionary algorithm for the flexible job shop scheduling problem. In *Proceedings of the AAAI conference on artificial intelligence: vol. 33* (pp. 2262–2271).
- Doh, H.-H., Yu, J.-M., Kim, J.-S., Lee, D.-H., & Nam, S.-H. (2013). A priority scheduling approach for flexible job shops with multiple process plans. *International Journal of Production Research*, 51(12), 3748–3764.
- Driemel, R., & Münch, L. (2012). An integrated scheduling and material-handling approach for complex job shops: A computational study. *International Journal of Production Research*, 50(20), 5966–5985.
- Driss, I., Mouss, K. N., & Laggoun, A. (2015). A new genetic algorithm for flexible job-shop scheduling problems. *Journal of Mechanical Science and Technology*, 29(3), 1273–1281.
- El Khoukhi, F., Boukachour, J., & Alaoui, A. E. H. (2017). The “dual-ants colony”: A novel hybrid approach for the flexible job shop scheduling problem with preventive maintenance. *Computers & Industrial Engineering*, 106, 236–255.
- Fan, J., Shen, W., Gao, L., Zhang, C., & Zhang, Z. (2021). A hybrid Jaya algorithm for solving flexible job shop scheduling problem considering multiple critical paths. *Journal of Manufacturing Systems*, 60, 298–311.
- Fattahi, P., & Fallahi, A. (2010). Dynamic scheduling in flexible job shop systems by considering simultaneously efficiency and stability. *CIRP Journal of Manufacturing Science and Technology*, 2(2), 114–123.
- Fattahi, P., Saidi Mehrabad, M., & Jolai, F. (2007). Mathematical modeling and heuristic approaches to flexible job shop scheduling problems. *Journal of Intelligent Manufacturing*, 18(3), 331–342.
- Flores Gómez, M., Borodin, V., & Dauzère-Pérès, S. (2021). A monte carlo based method to maximize the service level on the makespan in the stochastic flexible job-shop scheduling problem. In *2021 IEEE 17th international conference on automation science and engineering (CASE)* (pp. 2072–2077). IEEE.
- Fontes, D., Homayouni, S. M., et al., (2019). Joint production and transportation scheduling in flexible manufacturing systems. *Journal of Global Optimization*, 74(4), 879–908.
- French, S. (1982). *Sequencing and scheduling - an introduction to the mathematics of the job shop*. New York: Wiley.
- Frutos, M., Olivera, A. C., & Tohmé, F. (2010). A memetic algorithm based on a NS-GAI scheme for the flexible job-shop scheduling problem. *Annals of Operations Research*, 181(1), 745–765.
- Gao, J., Gen, M., & Sun, L. (2006). Scheduling jobs and maintenances in flexible job shop with a hybrid genetic algorithm. *Journal of Intelligent Manufacturing*, 17(4), 493–507.
- Gao, J., Sun, L., & Gen, M. (2008). A hybrid genetic and variable neighborhood descent algorithm for flexible job shop scheduling problems. *Computers and Operations Research*, 35(9), 2892–2907.
- Gao, K. Z., Suganthan, P. N., Pan, Q. K., Chua, T. J., Cai, T. X., & Chong, C. S. (2014). Pareto-based grouping discrete harmony search algorithm for multi-objective flexible job shop scheduling. *Information Sciences*, 289(1), 76–90.
- Gao, K. Z., Suganthan, P. N., Tasgetiren, M. F., Pan, Q. K., & Sun, Q. Q. (2015). Effective ensembles of heuristics for scheduling flexible job shop problem with new job insertion. *Computers and Industrial Engineering*, 90, 107–117.
- García-León, A. A., Dauzère-Pérès, S., & Mati, Y. (2019). An efficient Pareto approach for solving the multi-objective flexible job-shop scheduling problem with regular criteria. *Computers and Operations Research*, 108, 187–200.
- Gomes, M. C., Barbosa-Póvoa, A. P., & Novais, A. Q. (2005). Optimal scheduling for flexible job shop operation. *International Journal of Production Research*, 43(11), 2323–2353.
- Gomes, M. C., Barbosa-Póvoa, A. P., & Novais, A. Q. (2013). Reactive scheduling in a make-to-order flexible job shop with re-entrant process and assembly: A mathematical programming approach. *International Journal of Production Research*, 51(17), 5120–5141.
- González, M. A., Vela, C. R., & Varela, R. (2015). Scatter search with path relinking for the flexible job shop scheduling problem. *European Journal of Operational Research*, 245(1), 35–45.
- Graham, R. L., Lawler, E. L., Lenstra, J. K., & Kan, A. R. (1979). Optimization and approximation in deterministic sequencing and scheduling: A survey. In *Annals of discrete mathematics: vol. 5* (pp. 287–326). Elsevier.
- Grobler, J., Engelbrecht, A. P., Kok, S., & Yadavalli, S. (2010). Metaheuristics for the multi-objective FJSP with sequence-dependent set-up times, auxiliary resources and machine down time. *Annals of Operations Research*, 180(1), 165–196.
- Gröflin, H., & Klinkert, A. (2006). Feasible insertions in job shop scheduling, short cycles and stable sets. *European Journal of Operational Research*, 177(2), 763–785.
- Gröflin, H., Pham, D. N., & Bürgy, R. (2011). The flexible blocking job shop with transfer and set-up times. *Journal of Combinatorial Optimization*, 22(2), 121–144.
- Gu, X.-L., Huang, M., & Liang, X. (2020). A discrete particle swarm optimization algorithm with adaptive inertia weight for solving multiobjective flexible job-shop scheduling problem. *IEEE Access*, 8, 33125–33136.
- Hajibabaei, M., & Behnamian, J. (2021). Flexible job-shop scheduling problem with unrelated parallel machines and resources-dependent processing times: A tabu search algorithm. *International Journal of Management Science and Engineering Management*, 16(4), 242–253.
- Ham, A. (2020). Transfer-robot task scheduling in flexible job shop. *Journal of Intelligent Manufacturing*, 31(7), 1783–1793.
- Ham, A. M., & Cakici, E. (2016). Flexible job shop scheduling problem with parallel batch processing machines: MIP and CP approaches. *Computers and Industrial Engineering*, 102, 160–165.
- Hansmann, R. S., Rieger, T., & Zimmermann, U. T. (2014). Flexible job shop scheduling with blockages. *Mathematical Methods of Operations Research*, 79(2), 135–161.
- Hmida, A. B., Haouari, M., Huguet, M.-J., & Lopez, P. (2010). Discrepancy search for the flexible job shop scheduling problem. *Computers and Operations Research*, 37(12), 2192–2201.
- Homayouni, S. M., & Fontes, D. B. (2021). Production and transport scheduling in flexible job shop manufacturing systems. *Journal of Global Optimization*, 79(2), 463–502.
- Huang, B., Sun, Y., & Sun, Y. (2008). Scheduling of flexible manufacturing systems based on petri nets and hybrid heuristic search. *International Journal of Production Research*, 46(16), 4553–4565.
- Huang, R. H., Yang, C. L., & Cheng, W. C. (2013). Flexible job shop scheduling with due window—A two-pheromone ant colony approach. *International Journal of Production Economics*, 141(2), 685–697.
- Hurink, J., Jurisch, B., & Thole, M. (1994). Tabu search for the job-shop scheduling problem with multi-purpose machines. *OR Spektrum*, 15, 205–215.
- Ivens, P., Lambrecht, M., et al., (1996). Extending the shifting bottleneck procedure to real-life applications. *European Journal of Operational Research*, 90(2), 252–268.

- Jamrus, T., Chien, C.-F., Gen, M., & Sethanan, K. (2018). Hybrid particle swarm optimization combined with genetic operators for flexible job-shop scheduling under uncertain processing time for semiconductor manufacturing. *IEEE Transactions on Semiconductor Manufacturing*, 31(1), 32–41.
- Jeong, K.-C., & Kim, Y.-D. (1998). A real-time scheduling mechanism for a flexible manufacturing system: Using simulation and dispatching rules. *International Journal of Production Research*, 36(9), 2609–2626.
- Jiang, T., & Zhang, C. (2018). Application of grey wolf optimization for solving combinatorial problems: Job shop and flexible job shop scheduling cases. *IEEE Access*, 6, 26231–26240.
- Jurisch, B. (1995). Lower bounds for the job-shop scheduling problem on multi-purpose machines. *Discrete Applied Mathematics*, 58(2), 145–156.
- Kacem, I., Hammadi, S., & Borne, P. (2002a). Approach by localization and multi-objective evolutionary optimization for flexible job shop scheduling problems. *IEEE Transactions on Systems, Man, and Cybernetics*, 32, 1–13.
- Kacem, I., Hammadi, S., & Borne, P. (2002b). Pareto-optimality approach for flexible job-shop scheduling problems: Hybridization of evolutionary algorithms and fuzzy logic. *Mathematics and Computers in Simulation*, 60(3–5), 245–276.
- Karimi, H., Rahmati, S. H. A., & Zandieh, M. (2012). An efficient knowledge-based algorithm for the flexible job shop scheduling problem. *Knowledge-Based Systems*, 36, 236–244.
- Karimi, S., Ardalan, Z., Naderi, B., & Mohammadi, M. (2017). Scheduling flexible job-shops with transportation times: Mathematical models and a hybrid imperialist competitive algorithm. *Applied mathematical modelling*, 41, 667–682.
- Kasapidis, G. A., Dauzère-Pérès, S., Paraskevopoulos, D. C., Repoussis, P. P., & Tarantilis, C. D. (2023). On the multi-resource flexible job-shop scheduling problem with arbitrary precedence graphs. *Production and Operations Management*.
- Kasapidis, G. A., Paraskevopoulos, D. C., Repoussis, P. P., & Tarantilis, C. D. (2021). Flexible job shop scheduling problems with arbitrary precedence graphs. *Production and Operations Management*, 30(11), 4044–4068.
- Kato, E. R. R., de Aguiar Aranha, G. D., & Tsunaki, R. H. (2018). A new approach to solve the flexible job shop problem based on a hybrid particle swarm optimization and random-restart hill climbing. *Computers and Industrial Engineering*, 125, 178–189.
- Kemmoé-Tchomté, S., Lamy, D., & Tchernev, N. (2017). An effective multi-start multi-level evolutionary local search for the flexible job-shop problem. *Engineering Applications of Artificial Intelligence*, 62(January), 80–95.
- Kim, Y.-D., & Yano, C. A. (1994). A new branch and bound algorithm for loading problems in flexible manufacturing systems. *International Journal of Flexible Manufacturing Systems*, 6(4), 361–381.
- Kis, T. (2003). Job-shop scheduling with processing alternatives. *European Journal of Operational Research*, 151(2), 307–332.
- Knopp, S., Dauzère-Pérès, S., & Yugma, C. (2014). Flexible job-shop scheduling with extended route flexibility for semiconductor manufacturing. In *Proceedings of the winter simulation conference 2014* (pp. 2478–2489). IEEE.
- Knopp, S., Dauzère-Pérès, S., & Yugma, C. (2017). A batch-oblivious approach for complex job-shop scheduling problems. *European Journal of Operational Research*, 263(1), 50–61.
- Kubiak, W., Feng, Y., Li, G., Sethi, S. P., & Sriskandarajah, C. (2020). Efficient algorithms for flexible job shop scheduling with parallel machines. *Naval Research Logistics*, 67(4), 272–288.
- Kumar, M. V., Janardhana, R., & Rao, C. (2011). Simultaneous scheduling of machines and vehicles in an FMS environment with alternative routing. *The International Journal of Advanced Manufacturing Technology*, 53(1), 339–351.
- Lee, J., & Lee, J. S. (2010). Heuristic search for scheduling flexible manufacturing systems using lower bound reachability matrix. *Computers & Industrial Engineering*, 59(4), 799–806.
- Lee, S., Moon, I., Bae, H., & Kim, J. (2012). Flexible job-shop scheduling problems with 'AND'/'OR' precedence constraints. *International Journal of Production Research*, 50(7), 1979–2001.
- Lei, D. (2010). A genetic algorithm for flexible job shop scheduling with fuzzy processing time. *International Journal of Production Research*, 48(10), 2995–3013.
- Lei, D., & Guo, X. (2014). Variable neighbourhood search for dual-resource constrained flexible job shop scheduling. *International Journal of Production Research*, 52(9), 2519–2529.
- Li, J.-q., Deng, J.-w., Li, C.-y., Han, Y.-y., Tian, J., Zhang, B., & Wang, C.-g. (2020). An improved Jaya algorithm for solving the flexible job shop scheduling problem with transportation and setup times. *Knowledge-Based Systems*, 200, 106032.
- Li, X., & Gao, L. (2016). An effective hybrid genetic algorithm and tabu search for flexible job shop scheduling problem. *International Journal of Production Economics*, 174, 93–110.
- Li, Z. C., Qian, B., Hu, R., Chang, L. L., & Yang, J. B. (2019). An elitist nondominated sorting hybrid algorithm for multi-objective flexible job-shop scheduling problem with sequence-dependent setups. *Knowledge-Based Systems*, 173, 83–112.
- Lima, A., Borodin, V., Dauzère-Pérès, S., & Vialletelle, P. (2021). A sampling-based approach for managing lot release in time constraint tunnels in semiconductor manufacturing. *International Journal of Production Research*, 59(3), 860–884.
- Liu, S. Q., & Kozan, E. (2009). Scheduling trains as a blocking parallel-machine job shop scheduling problem. *Computers & Operations Research*, 36(10), 2840–2852.
- Liu, S. Q., & Kozan, E. (2011). Scheduling trains with priorities: A no-wait blocking parallel-machine job-shop scheduling model. *Transportation Science*, 45(2), 175–198.
- Liu, S. Q., & Kozan, E. (2012). A hybrid shifting bottleneck procedure algorithm for the parallel-machine job-shop scheduling problem. *Journal of the Operational Research Society*, 63(2), 168–182.
- Liu, Z., Wang, J., Zhang, C., Chu, H., Ding, G., & Zhang, L. (2021). A hybrid genetic-particle swarm algorithm based on multilevel neighbourhood structure for flexible job shop scheduling problem. *Computers and Operations Research*, 135, 105431.
- Lloyd, S., Yu, H., & Konstant, N. (1995). FMS scheduling using petri net modeling and a branch & bound search. In *Proceedings. IEEE international symposium on assembly and task planning* (pp. 141–146). IEEE.
- Loukil, T., Teghem, J., & Fortemps, P. (2007). A multi-objective production scheduling case study solved by simulated annealing. *European Journal of Operational Research*, 179(3), 709–722.
- Lu, C., Li, X., Gao, L., Liao, W., & Yi, J. (2017). An effective multi-objective discrete virus optimization algorithm for flexible job-shop scheduling problem with controllable processing times. *Computers and Industrial Engineering*, 104, 156–174.
- Lunardi, W. T., Birgin, E. G., Laborie, P., Ronconi, D. P., & Voos, H. (2020). Mixed integer linear programming and constraint programming models for the on-line printing shop scheduling problem. *Computers and Operations Research*, 123, 105020.
- Lunardi, W. T., Birgin, E. G., Ronconi, D. P., & Voos, H. (2021). Metaheuristics for the online printing shop scheduling problem. *European Journal of Operational Research*, 293(2), 419–441.
- Mahdavi, I., Shirazi, B., & Solimanpur, M. (2010). Development of a simulation-based decision support system for controlling stochastic flexible job shop manufacturing systems. *Simulation Modelling Practice and Theory*, 18(6), 768–786.
- Mascis, A., & Pacciarelli, D. (2002). Job-shop scheduling with blocking and no-wait constraints. *European Journal of Operational Research*, 143(3), 498–517.
- Mason, S. J., Fowler, J. W., Carlyle, W. M., & Montgomery, D. C. (2005). Heuristics for minimizing total weighted tardiness in complex job shops. *International Journal of Production Research*, 43(10), 1943–1963.
- Mason, S. J., Fowler, J. W., & Matthew Carlyle, W. (2002). A modified shifting bottleneck heuristic for minimizing total weighted tardiness in complex job shops. *Journal of Scheduling*, 5(3), 247–262.
- Mastrolilli, M., & Gambardella, L. M. (2000). Effective neighbourhood functions for the flexible job shop problem. *Journal of Scheduling*, 3(1), 3–20.
- Mati, Y., Lahlou, C., & Dauzère-Pérès, S. (2011). Modelling and solving a practical flexible job shop scheduling problem with blocking constraints. *International Journal of Production Research*, 49(8), 2169–2182.
- Mati, Y., Rezg, N., & Xie, X. (2001). An integrated greedy heuristic for a flexible job shop scheduling problem. In *2001 IEEE international conference on systems, man and cybernetics. e-systems and e-man for cybernetics in cyberspace* (Cat. no. 01CH37236): vol. 4 (pp. 2534–2539). IEEE.
- Mati, Y., & Xie, X. (2011). Multiresource shop scheduling with resource flexibility and blocking. *IEEE Transactions on Automation Science and Engineering*, 8(1), 175–189.
- Mejia, G., & Odrey, N. G. (2005). An approach using petri nets and improved heuristic search for manufacturing system scheduling. *Journal of Manufacturing Systems*, 24(2), 79–92.
- Meng, L., Zhang, C., Ren, Y., Zhang, B., & Lv, C. (2020). Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem. *Computers and Industrial Engineering*, 142, 106347.
- Meng, T., Pan, Q. K., & Sang, H. Y. (2018). A hybrid artificial bee colony algorithm for a flexible job shop scheduling problem with overlapping in operations. *International Journal of Production Research*, 56(16), 5278–5292.
- Mohammadi, S., Al-e Hashem, S. M. J. M., & Rekik, Y. (2020). An integrated production scheduling and delivery route planning with multi-purpose machines: A case study from a furniture manufacturing company. *International Journal of Production Economics*, 219(May 2019), 347–359.
- Mokhtari, H., & Dadgar, M. (2015). Scheduling optimization of a stochastic flexible job shop system with time-varying machine failure rate. *Computers and Operations Research*, 61, 31–45.
- Mokhtari, H., & Hasani, A. (2017). An energy-efficient multi-objective optimization for flexible job-shop scheduling problem. *Computers and Chemical Engineering*, 104, 339–352.
- Mönch, L., & Drießel, R. (2005). A distributed shifting bottleneck heuristic for complex job shops. *Computers and Industrial Engineering*, 49(3), 363–380.
- Mönch, L., Fowler, J. W., Dauzère-Pérès, S., Mason, S. J., Rose, O., et al., (2011). A survey of problems, solution techniques, and future challenges in scheduling semiconductor manufacturing operations. *Journal of Scheduling*, 14(6), 583–599.
- Mönch, L., Schabacker, R., Pabst, D., & Fowler, J. W. (2007). Genetic algorithm-based subproblem solution procedures for a modified shifting bottleneck heuristic for complex job shops. *European Journal of Operational Research*, 177(3), 2100–2118.
- Moon, I., Lee, S., & Bae, H. (2008). Genetic algorithms for job shop scheduling problems with alternative routings. *International Journal of Production Research*, 46(10), 2695–2705.
- Mousakhani, M. (2013). Sequence-dependent setup time flexible job shop scheduling problem to minimise total tardiness. *International Journal of Production Research*, 51(12), 3476–3487.
- Naderi, B., & Roshanaei, V. (2022). Critical-path-search logic-based benders decomposition approaches for flexible job shop scheduling. *INFORMS Journal on Optimization*, 4(1), 1–28.
- Neumann, K., Schwindt, C., & Zimmermann, J. (2003). *Project scheduling with time windows and scarce resources: Temporal and resource-constrained project scheduling with regular and nonregular objective functions*. Springer Science & Business Media.

- Nouri, H. E., Driss, O. B., & Ghédira, K. (2016). Simultaneous scheduling of machines and transport robots in flexible job shop environment using hybrid metaheuristics based on clustered holonic multiagent model. *Computers & Industrial Engineering*, 102, 488–501.
- Nowicki, E., & Smutnicki, C. (1996). A fast taboo search algorithm for the job shop problem. *Management Science*, 42(6), 797–813.
- O'Keefe, R. M., & Kasirajan, T. (1992). Interaction between dispatching and next station selection rules in a dedicated flexible manufacturing system. *International Journal of Production Research*, 30(8), 1753–1772.
- Ortiz, M. A., Betancourt, L. E., Negrete, K. P., De Felice, F., & Petrillo, A. (2018). Dispatching algorithm for production programming of flexible job-shop systems in the smart factory industry. *Annals of Operations Research*, 264(1), 409–433.
- Ovacik, I. M., & Uzsoy, R. (2012). *Decomposition methods for complex factory scheduling problems*. Springer Science & Business Media.
- Özgüven, C., Özbakir, L., & Yavuz, Y. (2010). Mathematical models for job-shop scheduling problems with routing and process plan flexibility. *Applied Mathematical Modelling*, 34(6), 1539–1548.
- Özgüven, C., Yavuz, Y., & Özbakir, L. (2012). Mixed integer goal programming models for the flexible job-shop scheduling problems with separable and non-separable sequence dependent setup times. *Applied Mathematical Modelling*, 36(2), 846–858.
- Ozturk, G., Bahadir, O., & Teymourifar, A. (2019). Extracting priority rules for dynamic multi-objective flexible job shop scheduling problems using gene expression programming. *International Journal of Production Research*, 57(10), 3121–3137.
- Palacios, J. J., González, M. A., Vela, C. R., González-Rodríguez, I., & Puente, J. (2015). Genetic tabu search for the fuzzy flexible job shop problem. *Computers and Operations Research*, 54, 74–89.
- Park, M.-J., & Ham, A. (2022). Energy-aware flexible job shop scheduling under time-of-use pricing. *International Journal of Production Economics*, 248(April), 108507.
- Pei, Z., Zhang, X., Zheng, L., & Wan, M. (2020). A column generation-based approach for proportionate flexible two-stage no-wait job shop scheduling. *International Journal of Production Research*, 58(2), 487–508.
- Pérez, M. A. F., & Raupp, F. M. P. (2016). A newton-based heuristic algorithm for multi-objective flexible job-shop scheduling problem. *Journal of Intelligent Manufacturing*, 27(2), 409–416.
- Pezzella, F., Morganti, G., & Ciaschetti, G. (2008). A genetic algorithm for the flexible job-shop scheduling problem. *Computers and Operations Research*, 35(10), 3202–3212.
- Pfund, M. E., Balasubramanian, H., Fowler, J. W., Mason, S. J., & Rose, O. (2008). A multi-criteria approach for scheduling semiconductor wafer fabrication facilities. *Journal of Scheduling*, 11(1), 29–47.
- Pham, D. N., & Klinkert, A. (2008). Surgical case scheduling as a generalized job shop scheduling problem. *European Journal of Operational Research*, 185(3), 1011–1025.
- Piroozfard, H., Wong, K. Y., & Wong, W. P. (2018). Minimizing total carbon footprint and total late work criterion in flexible job shop scheduling by using an improved multi-objective genetic algorithm. *Resources, Conservation and Recycling*, 128, 267–283.
- Poppenborg, J., Knust, S., & Hertzberg, J. (2012). Online scheduling of flexible job-shops with blocking and transportation. *European Journal of Industrial Engineering*, 6(4), 497–518.
- Prot, D., & Bellenguez-Morineau, O. (2012). Tabu search and lower bound for an industrial complex shop scheduling problem. *Computers and Industrial Engineering*, 62(4), 1109–1118.
- Quinton, F., Hamaz, I., & Houssin, L. (2020). A mixed integer linear programming modelling for the flexible cyclic jobshop problem. *Annals of Operations Research*, 285(1–2), 335–352.
- Raaymakers, W. H. M., & Hoogeveen, J. A. (2000). Scheduling multipurpose batch process industries with no-wait restrictions by simulated annealing. *European Journal of Operational Research*, 126(1), 131–151.
- Rajkumar, M., Asokan, P., & Vamsikrishna, V. (2010). A GRASP algorithm for flexible job-shop scheduling with maintenance constraints. *International Journal of Production Research*, 48(22), 6821–6836.
- Roshanaei, V., Azab, A., & ElMaraghy, H. (2013). Mathematical modelling and a meta-heuristic for flexible job shop scheduling. *International Journal of Production Research*, 51(20), 6247–6274.
- Rossi, A. (2014). Flexible job shop scheduling with sequence-dependent setup and transportation times by ant colony with reinforced pheromone relationships. *International Journal of Production Economics*, 153, 253–267.
- Rossi, A., & Dini, G. (2007). Flexible job-shop scheduling with routing flexibility and separable setup times using ant colony optimisation method. *Robotics and Computer-Integrated Manufacturing*, 23(5), 503–516.
- Schutten, J. M. J. (1998). Practical job shop scheduling. *Annals of Operations Research*, 83, 161–177.
- Shanker, K., & Tzen, Y.-J. (1985). A loading and dispatching problem in a random flexible manufacturing system. *International Journal of Production Research*, 23(3), 579–595.
- Shen, L., Dauzère-Pérès, S., & Neufeld, J. S. (2018). Solving the flexible job shop scheduling problem with sequence-dependent setup times. *European Journal of Operational Research*, 265(2), 503–516.
- Shen, L., Dauzère-Pérès, S., & Maeker, S. (2023). Energy cost efficient scheduling in flexible job-shop manufacturing systems. *European Journal of Operational Research*, 25pages.
- Shen, X. N., & Yao, X. (2015). Mathematical modeling and multi-objective evolutionary algorithms applied to dynamic flexible job shop scheduling problems. *Information Sciences*, 298(219), 198–224.
- Shi-Jin, W., Bing-Hai, Z., & Li-Feng, X. (2008). A filtered-beam-search-based heuristic algorithm for flexible job-shop scheduling problem. *International Journal of Production Research*, 46(11), 3027–3058.
- Sobeyko, O., & Mönch, L. (2016). Heuristic approaches for scheduling jobs in large-scale flexible job shops. *Computers and Operations Research*, 68, 97–109.
- Soto, C., Dorronsoro, B., Fraire, H., Cruz-Reyes, L., Gomez-Santillan, C., & Rangel, N. (2020). Solving the multi-objective flexible job shop scheduling problem with a novel parallel branch and bound algorithm. *Swarm and Evolutionary Computation*, 53(October 2019), Article 100632.
- Sourirajan, K., & Uzsoy, R. (2007). Hybrid decomposition heuristics for solving large-scale scheduling problems in semiconductor wafer fabrication. *Journal of Scheduling*, 10(1), 41–65.
- Sun, J., Zhang, G., Lu, J., & Zhang, W. (2021). A hybrid many-objective evolutionary algorithm for flexible job-shop scheduling problem with transportation and setup times. *Computers and Operations Research*, 132(January), Article 105263.
- Sundar, S., Suganthan, P. N., & Chua, T. J. (2013). A swarm intelligence approach to flexible job shop scheduling problem with no-wait constraint in remanufacturing. *Lecture notes in computer science (including subseries lecture notes in artificial intelligence and lecture notes in bioinformatics)*, 7895LNAI (PART 2), 593–602.
- Tamssaouet, K., & Dauzère-Pérès, S. (2023). A general efficient neighborhood structure framework for the job-shop and flexible job-shop scheduling problems. *European Journal of Operational Research*.
- Tamssaouet, K., Dauzère-Pérès, S., Knopp, S., Bitar, A., & Yugma, C. (2022). Multiobjective optimization for complex flexible job-shop scheduling problems. *European Journal of Operational Research*, 296(1), 87–100.
- Tamssaouet, K., Dauzère-Pérès, S., & Yugma, C. (2018). Metaheuristics for the job-shop scheduling problem with machine availability constraints. *Computers & Industrial Engineering*, 125, 1–8.
- Tay, J. C., & Ho, N. B. (2008). Evolving dispatching rules using genetic programming for solving multi-objective flexible job-shop problems. *Computers & Industrial Engineering*, 54(3), 453–473.
- Tayebi Araghi, M. E., Jolai, F., & Rabiee, M. (2014). Incorporating learning effect and deterioration for solving a SDST flexible job-shop scheduling problem with a hybrid meta-heuristic approach. *International Journal of Computer Integrated Manufacturing*, 27(8), 733–746.
- Thörnblad, K., Strömberg, A. B., Patriksson, M., & Almgren, T. (2015). Scheduling optimisation of a real flexible job shop including fixture availability and preventive maintenance. *European Journal of Industrial Engineering*, 9(1), 126–145.
- T'kindt, V., & Billaut, J.-C. (2006). *Multicriteria scheduling: Theory, models and algorithms*. Springer Science & Business Media.
- Türkylmaz, A., Senvar, O., Ünal, I., & Bulkan, S. (2020). A research survey: Heuristic approaches for solving multi objective flexible job shop problems. *Journal of Intelligent Manufacturing*, 31(8), 1949–1983.
- Türkylmaz, A., Senvar, O., Ünal, I., & Bulkan, S. (2022). A hybrid genetic algorithm based on a two-level hypervolume contribution measure selection strategy for bi-objective flexible job shop problem. *Computers and Operations Research*, 141(July 2021), Article 105694.
- Vilcot, G., & Billaut, J.-C. (2008). A tabu search and a genetic algorithm for solving a bicriteria general job shop scheduling problem. *European Journal of Operational Research*, 190(2), 398–411.
- Vilcot, G., & Billaut, J. C. (2011). A tabu search algorithm for solving a multicriteria flexible job shop scheduling problem. *International Journal of Production Research*, 49(23), 6963–6980.
- Wan, M., Fan, X., Zhang, F., & Bai, C. (2010). An integrated genetic algorithm for flexible job-shop scheduling problem. In *2010 international conference on computational intelligence and software engineering* (pp. 1–4). IEEE.
- Wang, L., Wang, S., Xu, Y., Zhou, G., & Liu, M. (2012a). A bi-population based estimation of distribution algorithm for the flexible job-shop scheduling problem. *Computers and Industrial Engineering*, 62(4), 917–926.
- Wang, L., Zhou, G., Xu, Y., & Liu, M. (2013). A hybrid artificial bee colony algorithm for the fuzzy flexible job-shop scheduling problem. *International Journal of Production Research*, 51(12), 3593–3608.
- Wang, L., Zhou, G., Xu, Y., Wang, S., & Liu, M. (2012b). An effective artificial bee colony algorithm for the flexible job-shop scheduling problem. *The International Journal of Advanced Manufacturing Technology*, 60(1–4), 303–315.
- Wang, S., & Yu, J. (2010). An effective heuristic for flexible job-shop scheduling problem with maintenance activities. *Computers and Industrial Engineering*, 59(3), 436–447.
- Wu, K., Huang, E., Wang, M., & Zheng, M. (2022). Job scheduling of diffusion furnaces in semiconductor fabrication facilities. *European Journal of Operational Research*, 301(1), 141–152.
- Wu, X., & Sun, Y. (2018). A green scheduling algorithm for flexible job shop with energy-saving measures. *Journal of Cleaner Production*, 172, 3249–3264.
- Xia, W., & Wu, Z. (2005). An effective hybrid optimization approach for multi-objective flexible job-shop scheduling problems. *Computers and Industrial Engineering*, 48(2), 409–425.
- Xing, L.-N., Chen, Y.-W., Wang, P., Zhao, Q.-S., & Xiong, J. (2010). A knowledge-based ant colony optimization for flexible job shop scheduling problems. *Applied Soft Computing*, 10(3), 888–896.
- Xiong, J., Xing, L. N., & Chen, Y. W. (2013). Robust scheduling for multi-objective flexible job-shop problems with random machine breakdowns. *International Journal of Production Economics*, 141(1), 112–126.

- Yazdani, M., Amiri, M., & Zandieh, M. (2010). Flexible job-shop scheduling with parallel variable neighborhood search algorithm. *Expert Systems with Applications*, 37(1), 678–687.
- Yazdani, M., Gholami, M., Zandieh, M., & Mousakhani, M. (2009). A simulated annealing algorithm for flexible job-shop scheduling problem. *Journal of Applied Sciences*, 9(4), 662–670.
- Yuan, Y., & Xu, H. (2013a). Flexible job shop scheduling using hybrid differential evolution algorithms. *Computers and Industrial Engineering*, 65(2), 246–260.
- Yuan, Y., & Xu, H. (2013b). An integrated search heuristic for large-scale flexible job shop scheduling problems. *Computers and Operations Research*, 40(12), 2864–2877.
- Yuan, Y., Xu, H., & Yang, J. (2013). A hybrid harmony search algorithm for the flexible job shop scheduling problem. *Applied Soft Computing*, 13(7), 3259–3272.
- Yugma, C., Dauzère-Pérès, S., Artigues, C., Derreumaux, A., Sibille, O., et al., (2012). A batching and scheduling algorithm for the diffusion area in semiconductor manufacturing. *International Journal of Production Research*, 50(8), 2118–2132.
- Zambrano Rey, G., Bekrar, A., Prabhu, V., & Trentesaux, D. (2014). Coupling a genetic algorithm with the distributed arrival-time control for the JIT dynamic scheduling of flexible job-shops. *International Journal of Production Research*, 52(12), 3688–3709.
- Zambrano Rey, G., Bekrar, A., Trentesaux, D., & Zhou, B. H. (2015). Solving the flexible job-shop just-in-time scheduling problem with quadratic earliness and tardiness costs. *International Journal of Advanced Manufacturing Technology*, 81(9–12), 1871–1891.
- Zhang, G., Gao, L., & Shi, Y. (2011). An effective genetic algorithm for the flexible job-shop scheduling problem. *Expert Systems with Applications*, 38(4), 3563–3573.
- Zhang, G., Shao, X., Li, P., & Gao, L. (2009). An effective hybrid particle swarm optimization algorithm for multi-objective flexible job-shop scheduling problem. *Computers and Industrial Engineering*, 56(4), 1309–1318.
- Zhang, J., & Yang, J. (2016). Flexible job-shop scheduling with flexible workdays, pre-emption, overlapping in operations and satisfaction criteria: An industrial application. *International Journal of Production Research*, 54(16), 4894–4918.
- Zhang, Q., Manier, H., & Manier, M.-A. (2012a). A genetic algorithm with tabu search procedure for flexible job shop scheduling with transportation constraints and bounded processing times. *Computers and Operations Research*, 39(7), 1713–1723.
- Zhang, Q., Manier, H., & Manier, M.-A. (2014). A modified shifting bottleneck heuristic and disjunctive graph for job shop scheduling problems with transportation constraints. *International Journal of Production Research*, 52(4), 985–1002.
- Zhang, R., Song, S., & Wu, C. (2012b). A two-stage hybrid particle swarm optimization algorithm for the stochastic job shop scheduling problem. *Knowledge-Based Systems*, 27, 393–406.
- Zhang, S., Li, X., Zhang, B., & Wang, S. (2020). Multi-objective optimisation in flexible assembly job shop scheduling using a distributed ant colony system. *European Journal of Operational Research*, 283(2), 441–460.
- Zhang, S., & Wang, S. (2018). Flexible assembly job-shop scheduling with sequence-dependent setup times and part sharing in a dynamic environment: Constraint programming model, mixed-integer programming model, and dispatching rules. *IEEE Transactions on Engineering Management*, 65, 487–504.
- Zhang, S., & Wong, T. N. (2017). Flexible job-shop scheduling/rescheduling in dynamic environment: A hybrid MAS/ACO approach. *International Journal of Production Research*, 55(11), 3173–3196.
- Zribi, N., El Kamel, A., & Borne, P. (2008). Minimizing the makespan for the MPM job-shop with availability constraints. *International Journal of Production Economics*, 112(1), 151–160.
- Zribi, N., Kacem, I., El Kamel, A., & Borne, P. (2007). Assignment and scheduling in flexible job-shops by hierarchical optimization. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 37(4), 652–661.